

# Modèles de calcul, complexité, logique

Loïck Lhote

`loick.lhote@unicaen.fr`



**UNIVERSITÉ  
CAEN  
NORMANDIE**



UNIVERSITÉ  
CAEN  
NORMANDIE

## 2- Introduction à l'algorithmique probabiliste



UNIVERSITÉ  
CAEN  
NORMANDIE

## 2- Introduction à l'algorithmique probabiliste

**Intérêts des algorithmes probabilistes**

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

- Multiples algorithmes probabilistes au quotidien
  - ☐ Jeux de hasard ou de stratégie sur ordinateur
  - ☐ Tests de programmes
  - ☐ Génération de clés de sécurité pour les communications
  - ☐ ...
- On ne peut être certain du résultat d'un algorithme déterministe

From [Wikipedia](#):

Studies by IBM in the 1990s suggest that computers typically experience about one cosmic-ray-induced error per 256 megabytes of RAM per month.<sup>[15]</sup>

This means a probability of  $3.7 \times 10^{-9}$  per byte per month, or  $1.4 \times 10^{-15}$  per byte per second. If your program runs for 1 minute and occupies 20 MB of RAM, then the failure probability would be

$$1 - (1 - 1.4e-15)^{60 \times 20 \times 1024^2} = 1.8e-6 \text{ a.k.a. "5 nines"}$$

Error checking can help to reduce the aftermath of failure. Also, because of more compact size of chips as commented by Joe, the failure rate could be different from what it was 20 years ago.

Source : <https://stackoverflow.com/questions/2580933/cosmic-rays-what-is-the-probability-they-will-affect-a-program>

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

*Exercice* : pour tester la primalité d'un nombre, nous disposons

- d'un algorithme probabiliste de complexité  $O(n^3)$  pour des entiers de  $n$  bits mais qui se trompe avec une probabilité de  $1/2^{100}$
- d'un algorithme déterministe de complexité  $O(n^6)$  pour des entiers de  $n$  bits
- les deux algorithmes utilisent 1Ko de mémoire
- d'un ordinateur pouvant effectuer  $10^{10}$  opérations par seconde (4 cœurs à 2.5GHz)

*Question* : Quel algorithme conseillez-vous pour  $n = 2048$  ?

Rappel : probabilité d'une erreur cosmique =  $1.47 \times 10^{-12}$  erreur/Ko/s

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

*Exercice* : pour tester la primalité d'un nombre, nous disposons

- d'un algorithme probabiliste de complexité  $O(n^3)$  pour des entiers de  $n$  bits mais qui se trompe avec une probabilité de  $1/2^{100}$
- d'un algorithme déterministe de complexité  $O(n^6)$  pour des entiers de  $n$  bits
- les deux algorithmes utilisent 1Ko de mémoire
- d'un ordinateur pouvant effectuer  $10^{10}$  opérations par seconde (4 cœurs à 2.5GHz)

*Question* : Quel algorithme conseillez-vous pour  $n = 2048$  ?

Rappel : probabilité d'une erreur cosmique =  $1.47 \times 10^{-12}$  erreur/Ko/s

alg. det. :  $\mathbb{P}(\text{erreur}) = 1 - (1 - 1.47 \times 10^{-12})^{2048^6/10^{10}} \approx 0.01$  (calcul : 360 ans!!!)

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

*Exercice* : pour tester la primalité d'un nombre, nous disposons

- d'un algorithme probabiliste de complexité  $O(n^3)$  pour des entiers de  $n$  bits mais qui se trompe avec une probabilité de  $1/2^{100}$
- d'un algorithme déterministe de complexité  $O(n^6)$  pour des entiers de  $n$  bits
- les deux algorithmes utilisent 1Ko de mémoire
- d'un ordinateur pouvant effectuer  $10^{10}$  opérations par seconde (4 cœurs à 2.5GHz)

*Question* : Quel algorithme conseillez-vous pour  $n = 2048$  ?

Rappel : probabilité d'une erreur cosmique =  $1.47 \times 10^{-12}$  erreur/Ko/s

alg. det. :  $\mathbb{P}(\text{erreur}) = 1 - (1 - 1.47 \times 10^{-12})^{2048^6/10^{10}} \approx 0.01$  (calcul : 360 ans!!!)

alg. prob. :  $\mathbb{P}(\text{erreur}) \approx 1 - (1 - 1.47 \times 10^{-12})^{2048^3/10^{10}} + 2^{-100} \approx 1.3 \times 10^{-12}$

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

*Question* : un tableau de  $n$  caractères ne contient que des  $a$  et des  $b$  et les  $a$  sont majoritaires. Donnez un algorithme qui étant donné un tel tableau, trouve la position d'un  $a$ .



# INTÉRÊTS DES ALGORITHMES PROBABILISTES

*Question* : un tableau de  $n$  caractères ne contient que des  $a$  et des  $b$  et les  $a$  sont majoritaires. Donnez un algorithme qui étant donné un tel tableau, trouve la position d'un  $a$ .

■ Solution 1 :

- ☐ on parcourt un par un les éléments du tableau
- ☐ complexité sur certaines entrées :  $O(n/2)$

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

*Question* : un tableau de  $n$  caractères ne contient que des  $a$  et des  $b$  et les  $a$  sont majoritaires. Donnez un algorithme qui étant donné un tel tableau, trouve la position d'un  $a$ .

## ■ Solution 1 :

- ☐ on parcourt un par un les éléments du tableau
- ☐ complexité sur certaines entrées :  $O(n/2)$

## ■ Solution 2 :

- ☐ tant qu'un  $a$  n'a pas été trouvé, choisir une case au hasard.
- ☐ complexité : 2 accès en moyenne quelle que soit l'entrée !!

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

## ■ Quitte à faire des concessions

- ☐ sur la certitude du temps de calcul et/ou
- ☐ sur la qualité du résultat

les algorithmes probabilistes sont

- ☐ souvent très efficaces en temps
- ☐ et souvent très précis dans les résultats

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

## ■ Quitte à faire des concessions

- ☐ sur la certitude du temps de calcul et/ou
- ☐ sur la qualité du résultat

les algorithmes probabilistes sont

- ☐ souvent très efficaces en temps
- ☐ et souvent très précis dans les résultats

**Difficulté :** concevoir et analyser ces algorithmes

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

## ■ Quitte à faire des concessions

- ☐ sur la certitude du temps de calcul et/ou
- ☐ sur la qualité du résultat

les algorithmes probabilistes sont

- ☐ souvent très efficaces en temps
- ☐ et souvent très précis dans les résultats

**Difficulté :** concevoir et analyser ces algorithmes

## ■ Notez que la notion de sécurité en cryptologie est naturellement basée sur les algorithmes probabilistes :

*“avoir une chance non négligeable de cracker un protocole”*

# INTÉRÊTS DES ALGORITHMES PROBABILISTES

## ■ Quitte à faire des concessions

- ☐ sur la certitude du temps de calcul et/ou
- ☐ sur la qualité du résultat

les algorithmes probabilistes sont

- ☐ souvent très efficaces en temps
- ☐ et souvent très précis dans les résultats

**Difficulté :** concevoir et analyser ces algorithmes

## ■ Notez que la notion de sécurité en cryptologie est naturellement basée sur les algorithmes probabilistes :

*“avoir une chance non négligeable de cracker un protocole”*

## ■ L'objectif est de présenter plusieurs familles d'algorithmes probabilistes

# PARADOXE DES ANNIVERSAIRES

Amusons nous un peu...

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

# PARADOXE DES ANNIVERSAIRES

Amusons nous un peu...

**Question** : Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse** : il faudra attendre la partie *Algorithmes Las-Vegas...*

**Pari** : Qui veut parier avec moi ?





UNIVERSITÉ  
CAEN  
NORMANDIE

## 2- Introduction à l'algorithmique probabiliste

**Notions basiques de probabilité**

On supposera connues les notions suivantes :

- distribution de probabilité,
- distribution uniforme,
- probabilités conditionnelles,
- variables aléatoires,
- espérance,
- variance ou écart-type,
- moments
- ...

# LOI DE BERNOULLI

## LOI DE BERNOULLI

Une variable aléatoire  $X$  suit une loi de Bernoulli de paramètre  $p \in [0, 1]$  si

$$\mathbb{P}[X = 1] = p, \quad \mathbb{P}[X = 0] = 1 - p.$$

L'espérance et la variance de  $X$  vérifient

$$\mathbb{E}[X] = p, \quad \mathbb{V}(X) = p(1 - p).$$

Exemples :

# LOI DE BERNOULLI

## LOI DE BERNOULLI

Une variable aléatoire  $X$  suit une loi de Bernoulli de paramètre  $p \in [0, 1]$  si

$$\mathbb{P}[X = 1] = p, \quad \mathbb{P}[X = 0] = 1 - p.$$

L'espérance et la variance de  $X$  vérifient

$$\mathbb{E}[X] = p, \quad \mathbb{V}(X) = p(1 - p).$$

Exemples :

■ lancé d'une pièce (biaisée ou non) : *pile* = 1 et *face* = 0

$[X = 1]$  = le lancé est tombé sur *pile*

# LOI DE BERNOULLI

## LOI DE BERNOULLI

Une variable aléatoire  $X$  suit une loi de Bernoulli de paramètre  $p \in [0, 1]$  si

$$\mathbb{P}[X = 1] = p, \quad \mathbb{P}[X = 0] = 1 - p.$$

L'espérance et la variance de  $X$  vérifient

$$\mathbb{E}[X] = p, \quad \mathbb{V}(X) = p(1 - p).$$

Exemples :

- lancé d'une pièce (biaisée ou non) : *pile* = 1 et *face* = 0

$[X = 1]$  = le lancé est tombé sur *pile*

- Un algorithme probabiliste donne la bonne (mauvaise) réponse avec une probabilité  $p$  quelle que soit l'entrée

# LOI DE BERNOULLI

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face ?

# LOI DE BERNOULLI

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

■ Quelle est la probabilité de faire face ?  $\mathbb{P}[face] = 1 - \frac{1}{3} = \frac{2}{3}$

# LOI DE BERNOULLI

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face?  $\mathbb{P}[face] = 1 - \frac{1}{3} = \frac{2}{3}$
- Quelle est la probabilité de faire successivement pile puis face?



Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face?  $\mathbb{P}[face] = 1 - \frac{1}{3} = \frac{2}{3}$
- Quelle est la probabilité de faire successivement pile puis face?

$$\mathbb{P}[pile - face] = \mathbb{P}[pile]\mathbb{P}[face] = \frac{1}{3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face ?  $\mathbb{P}[face] = 1 - \frac{1}{3} = \frac{2}{3}$
- Quelle est la probabilité de faire successivement pile puis face ?

$$\mathbb{P}[pile - face] = \mathbb{P}[pile]\mathbb{P}[face] = \frac{1}{3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

- Même question avec pile-pile-face-pile et face-pile-pile-pile ?

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face ?  $\mathbb{P}[face] = 1 - \frac{1}{3} = \frac{2}{3}$
- Quelle est la probabilité de faire successivement pile puis face ?

$$\mathbb{P}[pile-face] = \mathbb{P}[pile]\mathbb{P}[face] = \frac{1}{3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

- Même question avec pile-pile-face-pile et face-pile-pile-pile ?

$$\mathbb{P}[p-p-f-p] = \mathbb{P}[f-p-p-p] = \frac{1}{3^3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

# LOI DE BERNOULLI

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face ?  $\mathbb{P}[face] = 1 - \frac{1}{3} = \frac{2}{3}$
- Quelle est la probabilité de faire successivement pile puis face ?

$$\mathbb{P}[pile-face] = \mathbb{P}[pile]\mathbb{P}[face] = \frac{1}{3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

- Même question avec pile-pile-face-pile et face-pile-pile-pile ?

$$\mathbb{P}[p-p-f-p] = \mathbb{P}[f-p-p-p] = \frac{1}{3^3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

- En 4 lancés, quelle est la probabilité de faire 0,1,2,3,4 pile(s) ?

Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .

- Quelle est la probabilité de faire face ?  $\mathbb{P}[\text{face}] = 1 - \frac{1}{3} = \frac{2}{3}$
- Quelle est la probabilité de faire successivement pile puis face ?

$$\mathbb{P}[\text{pile} - \text{face}] = \mathbb{P}[\text{pile}] \mathbb{P}[\text{face}] = \frac{1}{3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

- Même question avec pile-pile-face-pile et face-pile-pile-pile ?

$$\mathbb{P}[p-p-f-p] = \mathbb{P}[f-p-p-p] = \frac{1}{3^3} \times \frac{2}{3} \quad (\text{lancés indépendants})$$

- En 4 lancés, quelle est la probabilité de faire 0,1,2,3,4 pile(s) ?

Loi binomiale de paramètres  $(4, 1/3)$  :

$$\mathbb{P}[k \text{ piles}] = \frac{4!}{k!(4-k)!} \left(\frac{1}{3}\right)^k \left(\frac{2}{3}\right)^{4-k}$$

# LOI BINOMIALE

## LOI BINOMIALE

Une variable aléatoire  $X$  suit une loi de binomiale de paramètres  $(n, p)$  avec  $n$  entiers et  $p \in [0, 1]$  si

$$\forall k \in \{1, 2, \dots, n\}, \quad \mathbb{P}[X = k] = \binom{n}{k} p^k (1 - p)^{n-k}.$$

L'espérance et la variance de  $X$  vérifient

$$\mathbb{E}[X] = np, \quad \mathbb{V}(X) = np(1 - p).$$

Exemples :

# LOI BINOMIALE

## LOI BINOMIALE

Une variable aléatoire  $X$  suit une loi de binomiale de paramètres  $(n, p)$  avec  $n$  entiers et  $p \in [0, 1]$  si

$$\forall k \in \{1, 2, \dots, n\}, \quad \mathbb{P}[X = k] = \binom{n}{k} p^k (1 - p)^{n-k}.$$

L'espérance et la variance de  $X$  vérifient

$$\mathbb{E}[X] = np, \quad \mathbb{V}(X) = np(1 - p).$$

### Exemples :

- Si  $X_1, X_2, \dots, X_n$  sont  $n$  variables aléatoires i.i.d. de même loi de Bernoulli de paramètre  $p$  alors

$$X_1 + X_2 + \dots + X_n \sim \text{Binom}(n, p).$$

# LOI BINOMIALE

## LOI BINOMIALE

Une variable aléatoire  $X$  suit une loi de binomiale de paramètres  $(n, p)$  avec  $n$  entiers et  $p \in [0, 1]$  si

$$\forall k \in \{1, 2, \dots, n\}, \quad \mathbb{P}[X = k] = \binom{n}{k} p^k (1 - p)^{n-k}.$$

L'espérance et la variance de  $X$  vérifient

$$\mathbb{E}[X] = np, \quad \mathbb{V}(X) = np(1 - p).$$

### Exemples :

- Si  $X_1, X_2, \dots, X_n$  sont  $n$  variables aléatoires i.i.d. de même loi de Bernoulli de paramètre  $p$  alors

$$X_1 + X_2 + \dots + X_n \sim \text{Binom}(n, p).$$

- lancé d'une pièce biaisée



- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$   
(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q1 : exécutons l'algorithme 1 fois sur une entrée donnée.  
Donner une minoration de la probabilité de succès de l'algorithme.

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$   
(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q1 : exécutons l'algorithme 1 fois sur une entrée donnée.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X$  la variable aléatoire qui vaut 1 si l'algorithme répond correctement et 0 sinon.

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$   
(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q1 : exécutons l'algorithme 1 fois sur une entrée donnée.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X$  la variable aléatoire qui vaut 1 si l'algorithme répond correctement et 0 sinon.

Alors  $X$  suit une loi de Bernoulli de paramètre  $p = 2/3$  et

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$   
(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q1 : exécutons l'algorithme 1 fois sur une entrée donnée.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X$  la variable aléatoire qui vaut 1 si l'algorithme répond correctement et 0 sinon.

Alors  $X$  suit une loi de Bernoulli de paramètre  $p = 2/3$  et

$$\mathbb{P}[\text{succes}] = \mathbb{P}[X = 1] = p = 2/3 \quad (\text{en fait } p \geq 2/3)$$

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$ .

(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q2 : Exécutons l'algorithme 3 fois sur une entrée donnée et retournons la réponse **majoritaire**.

Donner une minoration de la probabilité de succès de l'algorithme.

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$ .

(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q2 : Exécutons l'algorithme 3 fois sur une entrée donnée et retournons la réponse **majoritaire**.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X_i$  ( $i = 1, 2, 3$ ) la variable aléatoire qui vaut 1 si l'algorithme répond correctement à la  $i$ -ème exécution et 0 sinon.

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$ .

(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q2 : Exécutons l'algorithme 3 fois sur une entrée donnée et retournons la réponse **majoritaire**.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X_i$  ( $i = 1, 2, 3$ ) la variable aléatoire qui vaut 1 si l'algorithme répond correctement à la  $i$ -ème exécution et 0 sinon.

On pose  $Y = X_1 + X_2 + X_3$ .

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$ .

(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q2 : Exécutons l'algorithme 3 fois sur une entrée donnée et retournons la réponse **majoritaire**.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X_i$  ( $i = 1, 2, 3$ ) la variable aléatoire qui vaut 1 si l'algorithme répond correctement à la  $i$ -ème exécution et 0 sinon.

On pose  $Y = X_1 + X_2 + X_3$ .

Alors  $Y$  suit une loi binomiale de paramètres  $(3, p)$  et



- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$ .

(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q2 : Exécutons l'algorithme 3 fois sur une entrée donnée et retournons la réponse **majoritaire**.

Donner une minoration de la probabilité de succès de l'algorithme.

Réponse : on pose  $X_i$  ( $i = 1, 2, 3$ ) la variable aléatoire qui vaut 1 si l'algorithme répond correctement à la  $i$ -ème exécution et 0 sinon.

On pose  $Y = X_1 + X_2 + X_3$ .

Alors  $Y$  suit une loi binomiale de paramètres  $(3, p)$  et

$$\mathbb{P}[\text{succes}] = \mathbb{P}[Y \geq 2] = \binom{3}{2} p^2 (1 - p) + \binom{3}{3} p^3 = \frac{20}{27} \approx 0.74$$

- Considérons un problème de décision : la réponse est oui/non
- Considérons un algorithme qui pour toute entrée, retourne la bonne solution avec une probabilité  $p = 2/3$ .  
(On ne sait pas ce qu'il retourne avec probabilité  $1 - p = 1/3$ )

Q3 : Refaites le même exercice avec  $p = 1/2$  et  $p = 1/3$ . Que constatez-vous ?

- Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .
- Nous lançons la pièce jusqu'à obtenir face.

Q1 : Quelle est la probabilité de faire 1, 2, 3, 4 lancers.

- Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .
- Nous lançons la pièce jusqu'à obtenir face.

Q1 : Quelle est la probabilité de faire 1, 2, 3, 4 lancers.

Réponse : on pose  $X$  la v.a. égale au nombre de lancers.

$$\mathbb{P}[X = 1] = \mathbb{P}[face] = \frac{2}{3}, \quad \mathbb{P}[X = 2] = \mathbb{P}[pile - face] = \frac{1}{3} \times \frac{2}{3},$$

- Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .
- Nous lançons la pièce jusqu'à obtenir face.

Q1 : Quelle est la probabilité de faire 1, 2, 3, 4 lancers.

Réponse : on pose  $X$  la v.a. égale au nombre de lancers.

$$\mathbb{P}[X = 1] = \mathbb{P}[\text{face}] = \frac{2}{3}, \quad \mathbb{P}[X = 2] = \mathbb{P}[\text{pile} - \text{face}] = \frac{1}{3} \times \frac{2}{3},$$

$$\mathbb{P}[X = 3] = \mathbb{P}[\text{pile} - \text{pile} - \text{face}] = \frac{1}{3} \times \frac{1}{3} \times \frac{2}{3} = \left(\frac{1}{3}\right)^2 \cdot \frac{2}{3},$$

- Considérons une pièce biaisée dont la probabilité de faire pile est  $1/3$ .
- Nous lançons la pièce jusqu'à obtenir face.

Q1 : Quelle est la probabilité de faire 1, 2, 3, 4 lancers.

Réponse : on pose  $X$  la v.a. égale au nombre de lancers.

$$\mathbb{P}[X = 1] = \mathbb{P}[\text{face}] = \frac{2}{3}, \quad \mathbb{P}[X = 2] = \mathbb{P}[\text{pile} - \text{face}] = \frac{1}{3} \times \frac{2}{3},$$

$$\mathbb{P}[X = 3] = \mathbb{P}[\text{pile} - \text{pile} - \text{face}] = \frac{1}{3} \times \frac{1}{3} \times \frac{2}{3} = \left(\frac{1}{3}\right)^2 \cdot \frac{2}{3},$$

$$\mathbb{P}[X = 4] = \mathbb{P}[\text{pile} - \text{pile} - \text{pile} - \text{face}] = \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times \frac{2}{3} = \left(\frac{1}{3}\right)^3 \cdot \frac{2}{3}$$

## LOI GÉOMÉTRIQUE

Une variable aléatoire  $X$  suit une loi géométrique de paramètres  $p \in [0, 1]$  si

$$\forall k \geq 1, \quad \mathbb{P}[X = k] = p(1 - p)^{k-1}.$$

L'espérance et la variance de  $X$  vérifient (pour  $p \neq 0$ )

$$\mathbb{E}[X] = \frac{1}{p}, \quad \mathbb{V}(X) = \frac{1 - p}{p^2}.$$

Application : soit  $x$  un entier aléatoire de  $n$  bits. La probabilité que  $x$  soit premier est

$$\mathbb{P}[x \text{ est premier}] = \frac{1}{n \ln 2}.$$

## LOI GÉOMÉTRIQUE

Une variable aléatoire  $X$  suit une loi géométrique de paramètres  $p \in [0, 1]$  si

$$\forall k \geq 1, \quad \mathbb{P}[X = k] = p(1 - p)^{k-1}.$$

L'espérance et la variance de  $X$  vérifient (pour  $p \neq 0$ )

$$\mathbb{E}[X] = \frac{1}{p}, \quad \mathbb{V}(X) = \frac{1 - p}{p^2}.$$

Application : soit  $x$  un entier aléatoire de  $n$  bits. La probabilité que  $x$  soit premier est

$$\mathbb{P}[x \text{ est premier}] = \frac{1}{n \ln 2}.$$

On tire  $x$  jusqu'à obtenir un nombre premier. Quelle est la probabilité de faire 10 tirages ?



## LOI GÉOMÉTRIQUE

Une variable aléatoire  $X$  suit une loi géométrique de paramètres  $p \in [0, 1]$  si

$$\forall k \geq 1, \quad \mathbb{P}[X = k] = p(1 - p)^{k-1}.$$

L'espérance et la variance de  $X$  vérifient (pour  $p \neq 0$ )

$$\mathbb{E}[X] = \frac{1}{p}, \quad \mathbb{V}(X) = \frac{1 - p}{p^2}.$$

Application : soit  $x$  un entier aléatoire de  $n$  bits. La probabilité que  $x$  soit premier est

$$\mathbb{P}[x \text{ est premier}] = \frac{1}{n \ln 2}.$$

On tire  $x$  jusqu'à obtenir un nombre premier. Quelle est la probabilité de faire 10 tirages ? Quel est le nombre moyen de tirages ?

# INÉGALITÉ DE MARKOV

Soit  $X$  une v.a. prenant des valeurs positives dans un ensemble  $E$ . Soit  $a$  un réel strictement positif.

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

# INÉGALITÉ DE MARKOV

Soit  $X$  une v.a. prenant des valeurs positives dans un ensemble  $E$ . Soit  $a$  un réel strictement positif.

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

Preuve :

$$\begin{aligned}\mathbb{E}[X] &= \sum_{x \in E} x \mathbb{P}(X = x) \\ &= \sum_{x \in E, x < a} x \mathbb{P}(X = x) + \sum_{x \in E, x \geq a} x \mathbb{P}(X = x) \\ &\geq \sum_{x \in E, x < a} x \mathbb{P}(X = x) \\ &\geq a \sum_{x \in E, x < a} \mathbb{P}(X = x) \\ &\geq a \mathbb{P}(X \geq a)\end{aligned}$$

# INÉGALITÉ DE MARKOV

Soit  $X$  une v.a. prenant des valeurs positives dans un ensemble  $E$ . Soit  $a$  un réel strictement positif.

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

Preuve :

$$\begin{aligned}\mathbb{E}[X] &= \sum_{x \in E} x \mathbb{P}(X = x) \\ &= \sum_{x \in E, x < a} x \mathbb{P}(X = x) + \sum_{x \in E, x \geq a} x \mathbb{P}(X = x) \\ &\geq \sum_{x \in E, x < a} x \mathbb{P}(X = x) \\ &\geq a \sum_{x \in E, x < a} \mathbb{P}(X = x) \\ &\geq a \mathbb{P}(X \geq a)\end{aligned}$$

Remarque : si  $a \gg \mathbb{E}[X]$ , alors la probabilité tend vers 0. Il est donc peu probable que  $X$  soit d'ordre plus grand que son espérance

$$\text{Exemple : si } \mathbb{E}[X] \underset{n \rightarrow \infty}{\sim} n, \quad \mathbb{P}[X > n \ln \ln \ln n] \leq \frac{1}{\ln \ln \ln n} \underset{n \rightarrow \infty}{\rightarrow} 0$$

# INÉGALITÉ DE BIENAYMÉ-CHEBYCHEV

Soit  $X$  une variable aléatoire. Pour tout  $a > 0$ ,

$$\mathbb{P} (|X - \mathbb{E}[X]| \geq a) \leq \frac{\mathbb{V}(X)}{a^2}.$$

# INÉGALITÉ DE BIENAYMÉ-CHEBYCHEV

Soit  $X$  une variable aléatoire. Pour tout  $a > 0$ ,

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq a) \leq \frac{\mathbb{V}(X)}{a^2}.$$

Preuve :

Comme  $Y = (X - \mathbb{E}[X])^2$  est une v.a. prenant des valeurs positives ou nulles, on peut appliquer l'inégalité de Markov sur  $Y$ .

$$\mathbb{P}((X - \mathbb{E}[X])^2 \geq a^2) = \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{a^2} = \frac{\mathbb{V}(X)}{a^2}.$$

# INÉGALITÉ DE BIENAYMÉ-CHEBYCHEV

Soit  $X$  une variable aléatoire. Pour tout  $a > 0$ ,

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq a) \leq \frac{\mathbb{V}(X)}{a^2}.$$

Preuve :

Comme  $Y = (X - \mathbb{E}[X])^2$  est une v.a. prenant des valeurs positives ou nulles, on peut appliquer l'inégalité de Markov sur  $Y$ .

$$\mathbb{P}((X - \mathbb{E}[X])^2 \geq a^2) = \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{a^2} = \frac{\mathbb{V}(X)}{a^2}.$$

Remarque : si  $a^2 \gg \mathbb{V}(X)$ , alors la probabilité tend vers 0. Il est donc peu probable que  $X$  s'écarte de son espérance d'un ordre plus grand que sa variance

$$\text{Exemple : si } \mathbb{V}(X) \underset{n \rightarrow \infty}{\sim} n, \quad \mathbb{P}(|X - \mathbb{E}[X]| \geq \sqrt{n \ln \ln \ln n}) \leq \frac{1}{\ln \ln \ln n} \underset{n \rightarrow \infty}{\rightarrow} 0$$

# LOI DES GRANDS NOMBRES

## LOI DES GRANDS NOMBRES

Soit  $X_1, \dots, X_n$  des variables aléatoires indépendantes de même espérance (notée  $\mathbb{E}(X)$ ) et de même variance (notée  $\mathbb{V}(X)$ ). Alors,

$$\forall \epsilon > 0, \quad \mathbb{P} \left[ \left| \frac{X_1 + X_2 + \dots + X_n}{n} - \mathbb{E}[X] \right| > \epsilon \right] \xrightarrow{n \rightarrow \infty} 0.$$

- Autrement dit,  $(X_1 + X_2 + \dots + X_n)/n$  est un estimateur convergent de  $\mathbb{E}[X]$ .
- On peut affiner l'erreur commise en utilisant des résultats de grandes déviations



# LOI DES GRANDS NOMBRES

- On souhaite estimer une quantité  $r$ .

# LOI DES GRANDS NOMBRES

- On souhaite estimer une quantité  $r$ .
- On construit une expérience aléatoire  $X$  telle que  $\mathbb{E}[X] = r$

# LOI DES GRANDS NOMBRES

- On souhaite estimer une quantité  $r$ .
- On construit une expérience aléatoire  $X$  telle que  $\mathbb{E}[X] = r$
- On répète  $n$  fois l'expérience aléatoires  $X : X_1, X_2, \dots, X_n$

# LOI DES GRANDS NOMBRES

- On souhaite estimer une quantité  $r$ .
- On construit une expérience aléatoire  $X$  telle que  $\mathbb{E}[X] = r$
- On répète  $n$  fois l'expérience aléatoires  $X : X_1, X_2, \dots, X_n$
- Pour  $n$  assez grand, la loi des grands nombres donne

$$\frac{X_1 + X_2 + \dots + X_n}{n} \approx \mathbb{E}[X] = r \quad \text{au sens probabiliste}$$

# LOI DES GRANDS NOMBRES

- On souhaite estimer une quantité  $r$ .
- On construit une expérience aléatoire  $X$  telle que  $\mathbb{E}[X] = r$
- On répète  $n$  fois l'expérience aléatoires  $X : X_1, X_2, \dots, X_n$
- Pour  $n$  assez grand, la loi des grands nombres donne

$$\frac{X_1 + X_2 + \dots + X_n}{n} \approx \mathbb{E}[X] = r \quad \text{au sens probabiliste}$$

- Le résultat n'est pas exacte : une erreur (qui dépend de  $n$ ) est probablement commise



UNIVERSITÉ  
CAEN  
NORMANDIE

## 2- Introduction à l'algorithmique probabiliste

**Algorithmes de type Monté-Carlo**

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

- La distribution de probabilité de la réponse est telle qu'en itérant l'algorithme, un **biais** vers la bonne solution se produit



## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

- La distribution de probabilité de la réponse est telle qu'en itérant l'algorithme, un **biais** vers la bonne solution se produit
- On ne sait pas forcément déterminer si la réponse retournée est correcte ou non !

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 1 : considérons un problème de décision (la réponse est oui/non).  
Considérons l'algorithme qui avec une probabilité  $1/3$  retourne "oui" et une probabilité  $2/3$  retourne "non".

*Question : Est-ce un algorithme de Monte-Carlo ?*

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 1 : considérons un problème de décision (la réponse est oui/non).  
Considérons l'algorithme qui avec une probabilité  $1/3$  retourne "oui" et une probabilité  $2/3$  retourne "non".

*Question : Est-ce un algorithme de Monte-Carlo ?*

→ **NON !** En répétant l'algorithme, aucune information sur la réponse n'est apportée (grosso modo,  $1/3$  de "oui" et  $2/3$  de "non")

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 2 : considérons un problème de décision (la réponse est oui/non).  
Considérons l'algorithme qui avec une probabilité  $1/2$  retourne la bonne solution.

*Question : Est-ce un algorithme de Monte-Carlo ?*

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 2 : considérons un problème de décision (la réponse est oui/non).  
Considérons l'algorithme qui avec une probabilité  $1/2$  retourne la bonne solution.

*Question : Est-ce un algorithme de Monte-Carlo ?*

→ **NON !** En répétant l'algorithme, aucune information sur la réponse n'est apportée (grosso modo,  $1/2$  de "oui" et  $1/2$  de "non")

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.



## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 3 : considérons un problème de décision et un algorithme qui avec une probabilité  $\frac{1}{2} + \epsilon$  retourne la bonne solution.

*Question : Est-ce un algorithme de Monte-Carlo ?*

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 3 : considérons un problème de décision et un algorithme qui avec une probabilité  $\frac{1}{2} + \epsilon$  retourne la bonne solution.

*Question : Est-ce un algorithme de Monte-Carlo ?*

→ **OUI!** En répétant l'algorithme, la bonne réponse devient **majoritaire** (grosso modo,  $\frac{1}{2} + \epsilon$  de réponses correctes et  $\frac{1}{2} - \epsilon$  d'incorrectes)

## ALGORITHME DE MONTÉ-CARLO

Un algorithme de Monté-Carlo est un **algorithme probabiliste**

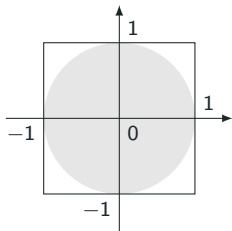
1. dont le temps d'exécution dans le pire des cas est déterministe (borne de complexité qui ne dépend pas de l'aléa),
2. dont le résultat peut être erroné selon une probabilité quantifiable,
3. dont on peut réduire l'erreur (en probabilité) en répétant l'algorithme.

Exemple 3 : considérons un problème de décision et un algorithme qui avec une probabilité  $\frac{1}{2} + \epsilon$  retourne la bonne solution.

*Question : Est-ce un algorithme de Monte-Carlo ?*

- **OUI !** En répétant l'algorithme, la bonne réponse devient **majoritaire** (grosso modo,  $\frac{1}{2} + \epsilon$  de réponses correctes et  $\frac{1}{2} - \epsilon$  d'incorrectes)
- **Le  $\epsilon$  est fondamental (biais)**

# ESTIMATION DE $\pi$

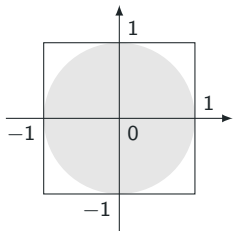


■ loi de  $X$  :

- L'aire du carré est  $\mathcal{A}(\text{carré}) = 4$
- L'aire du cercle est  $\mathcal{A}(\text{cercle}) = \pi$
- On tire uniformément  $(x, y)$  dans le carré
- On pose  $X$  la variable aléatoire

$$X = \begin{cases} 1 & \text{si } (x, y) \text{ est dans cercle} \\ 0 & \text{sinon} \end{cases}$$

# ESTIMATION DE $\pi$



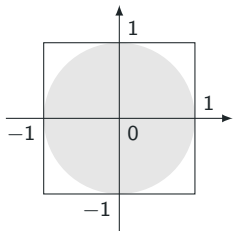
- L'aire du carré est  $\mathcal{A}(\text{carré}) = 4$
- L'aire du cercle est  $\mathcal{A}(\text{cercle}) = \pi$
- On tire uniformément  $(x, y)$  dans le carré
- On pose  $X$  la variable aléatoire

$$X = \begin{cases} 1 & \text{si } (x, y) \text{ est dans cercle} \\ 0 & \text{sinon} \end{cases}$$

- loi de  $X$  :  $X \sim \text{Bern}\left(\frac{\pi}{4}\right)$

$$\mathbb{P}[X = 1] = \frac{\pi}{4}, \quad \mathbb{P}[X = 0] = 1 - \frac{\pi}{4}, \quad \mathbb{E}[X] = \frac{\pi}{4}.$$

# ESTIMATION DE $\pi$



- L'aire du carré est  $\mathcal{A}(\text{carré}) = 4$
- L'aire du cercle est  $\mathcal{A}(\text{cercle}) = \pi$
- On tire uniformément  $(x, y)$  dans le carré
- On pose  $X$  la variable aléatoire

$$X = \begin{cases} 1 & \text{si } (x, y) \text{ est dans cercle} \\ 0 & \text{sinon} \end{cases}$$

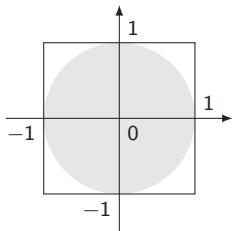
- loi de  $X$  :  $X \sim \text{Bern}\left(\frac{\pi}{4}\right)$

$$\mathbb{P}[X = 1] = \frac{\pi}{4}, \quad \mathbb{P}[X = 0] = 1 - \frac{\pi}{4}, \quad \mathbb{E}[X] = \frac{\pi}{4}.$$

- Loi des grands nombres : si  $X_1, \dots, X_n$  sont i.i.d de même loi  $\text{Bern}\left(\frac{\pi}{4}\right)$  alors

$$\frac{X_1 + \dots + X_n}{n} \xrightarrow[n \rightarrow \infty]{} \pi/4$$

# ESTIMATION DE $\pi$



- L'aire du carré est  $\mathcal{A}(\text{carré}) = 4$
- L'aire du cercle est  $\mathcal{A}(\text{cercle}) = \pi$
- On tire uniformément  $(x, y)$  dans le carré
- On pose  $X$  la variable aléatoire

$$X = \begin{cases} 1 & \text{si } (x, y) \text{ est dans cercle} \\ 0 & \text{sinon} \end{cases}$$

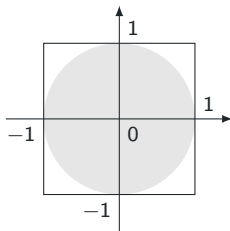
- loi de  $X$  :  $X \sim \text{Bern}\left(\frac{\pi}{4}\right)$

$$\mathbb{P}[X = 1] = \frac{\pi}{4}, \quad \mathbb{P}[X = 0] = 1 - \frac{\pi}{4}, \quad \mathbb{E}[X] = \frac{\pi}{4}.$$

- Loi des grands nombres : si  $X_1, \dots, X_n$  sont i.i.d de même loi  $\text{Bern}\left(\frac{\pi}{4}\right)$  alors

$$\frac{X_1 + \dots + X_n}{n} \xrightarrow[n \rightarrow \infty]{} \frac{\pi}{4} \quad \text{p.s.} \quad (\text{plus de détails en TD})$$

# ESTIMATION DE $\pi$



## Estimation de $\pi$

**Entrée :**  $n$  le nombre de tests

**Sortie :** Une estimation de  $\pi$  après  $n$  tests

1 : *compteur* = 0

2 : **Pour**  $i = 1$  à  $n$  **do**

3 :     choisir  $x$  et  $y$  uniformément dans  $[-1, 1]$

4 :     **Si**  $x^2 + y^2 \leq 1$  **Alors**

5 :         incrémenter *compteur* de 1

6 :     **Fin Si**

7 : **Fin Pour**

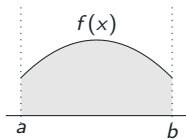
8 : **Retourner**  $\text{compteur} \times 4/n$

- Nous verrons en TD comment choisir  $n$  pour une précision donnée avec une probabilité fixée.



# ESTIMATION D'UNE INTÉGRALE

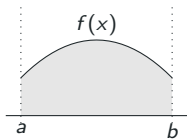
- On pose  $X$  la variable aléatoire uniforme sur  $[a, b]$
- On pose  $Y$  la variable aléatoire  $Y = f(X)$ .
- Nous avons



$$\mathbb{E}[Y] =$$

# ESTIMATION D'UNE INTÉGRALE

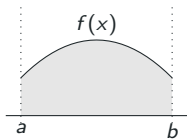
- On pose  $X$  la variable aléatoire uniforme sur  $[a, b]$
- On pose  $Y$  la variable aléatoire  $Y = f(X)$ .
- Nous avons



$$\mathbb{E}[Y] = \frac{1}{b-a} \int_a^b f(x) dx.$$

# ESTIMATION D'UNE INTÉGRALE

- On pose  $X$  la variable aléatoire uniforme sur  $[a, b]$
- On pose  $Y$  la variable aléatoire  $Y = f(X)$ .
- Nous avons



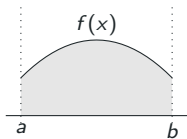
$$\mathbb{E}[Y] = \frac{1}{b-a} \int_a^b f(x) dx.$$

- Loi des grands nombres : soit  $X_1, \dots, X_n$  i.i.d de même loi uniforme sur  $[a, b]$ . On pose  $Y_i = f(X_i)$  pour  $i = 1 \dots n$ . Alors

$$\frac{Y_1 + \dots + Y_n}{n} \xrightarrow{n \rightarrow \infty}$$

# ESTIMATION D'UNE INTÉGRALE

- On pose  $X$  la variable aléatoire uniforme sur  $[a, b]$
- On pose  $Y$  la variable aléatoire  $Y = f(X)$ .
- Nous avons



$$\mathbb{E}[Y] = \frac{1}{b-a} \int_a^b f(x) dx.$$

- Loi des grands nombres : soit  $X_1, \dots, X_n$  i.i.d de même loi uniforme sur  $[a, b]$ . On pose  $Y_i = f(X_i)$  pour  $i = 1 \dots n$ . Alors

$$\frac{Y_1 + \dots + Y_n}{n} \xrightarrow[n \rightarrow \infty]{} \mathbb{E}[Y] = \frac{1}{b-a} \int_a^b f(x) dx.$$

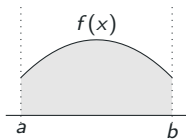
# ESTIMATION D'UNE INTÉGRALE

## Estimation d'une intégrale

**Entrée :** une fonction  $f$  sur  $[a, b]$ ,  
 $n$  le nombre de points dans l'intervalle  $[a, b]$

**Sortie :** Une estimation de  $\int_a^b f(x)dx$ .

```
1 :  $accu = 0$   
2 : Pour  $i = 1$  à  $n$  do  
3 :   choisir  $x \in [a, b]$   
4 :   ajouter  $f(x)$  à  $accu$   
5 : Fin Pour  
6 : Retourner  $(b - a) \times accu / n$ 
```



- Nous verrons en TD comment choisir  $n$  pour une précision donnée avec une probabilité fixée.
- L'algorithme fonctionne aussi pour les intégrales multiples !

# ÉLECTION D'UN LEADER DANS UN RÉSEAU

Un réseau est modélisé par un graphe **complet**  $G = (V, E)$  à  $n$  sommets.

On souhaite élire un leader dans le réseau (algorithme distribué)

# ÉLECTION D'UN LEADER DANS UN RÉSEAU

Un réseau est modélisé par un graphe **complet**  $G = (V, E)$  à  $n$  sommets.

On souhaite élire un leader dans le réseau (algorithme distribué)

## Élection d'un leader

**Entrée :** Partager un entier  $N$  avec ses voisins

1 : Chaque sommet  $v$  tire un entier  $r(v)$  uniformément dans  $[1, N]$

2 : Chaque sommet  $v$  transmet  $r(v)$  à ses voisins

3 : Pour un sommet  $v$ , **Si**  $r(v) > r(v')$  pour tous ses voisins  $v'$

**Alors**  $Etat_v = élu$

**Sinon**  $Etat_v = non - élu$

4 : Chaque sommet transmet son état à ses voisins et reçoit l'état de ses voisins

# ÉLECTION D'UN LEADER DANS UN RÉSEAU

Un réseau est modélisé par un graphe **complet**  $G = (V, E)$  à  $n$  sommets.

On souhaite élire un leader dans le réseau (algorithme distribué)

## Élection d'un leader

**Entrée :** Partager un entier  $N$  avec ses voisins

1 : Chaque sommet  $v$  tire un entier  $r(v)$  uniformément dans  $[1, N]$

2 : Chaque sommet  $v$  transmet  $r(v)$  à ses voisins

3 : Pour un sommet  $v$ , **Si**  $r(v) > r(v')$  pour tous ses voisins  $v'$

**Alors**  $Etat_v = élu$

**Sinon**  $Etat_v = non - élu$

4 : Chaque sommet transmet son état à ses voisins et reçoit l'état de ses voisins

■ L'algorithme fonctionne-t'il à tous les coups ?



# ÉLECTION D'UN LEADER DANS UN RÉSEAU

Un réseau est modélisé par un graphe **complet**  $G = (V, E)$  à  $n$  sommets.

On souhaite élire un leader dans le réseau (algorithme distribué)

## Élection d'un leader

**Entrée :** Partager un entier  $N$  avec ses voisins

1 : Chaque sommet  $v$  tire un entier  $r(v)$  uniformément dans  $[1, N]$

2 : Chaque sommet  $v$  transmet  $r(v)$  à ses voisins

3 : Pour un sommet  $v$ , **Si**  $r(v) > r(v')$  pour tous ses voisins  $v'$

**Alors**  $Etat_v = élu$

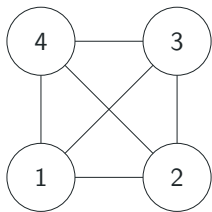
**Sinon**  $Etat_v = non - élu$

4 : Chaque sommet transmet son état à ses voisins et reçoit l'état de ses voisins

■ L'algorithme fonctionne-t'il à tous les coups ? Non !

# ÉLECTION D'UN LEADER DANS UN RÉSEAU

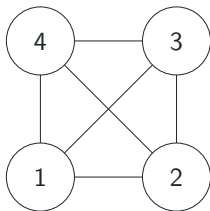
$$r(4) = 3 \quad r(3) = 2$$



$$r(1) = 8 \quad r(2) = 8$$

Pas d'élection !

$$r(4) = 3 \quad r(3) = 2$$



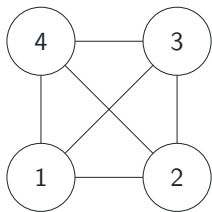
$$r(1) = 8 \quad r(2) = 7$$

Élection

■ Quelle est la probabilité de succès de l'algorithme (pour  $N = n$ ) ?

# ÉLECTION D'UN LEADER DANS UN RÉSEAU

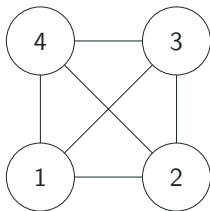
$$r(4) = 3 \quad r(3) = 2$$



$$r(1) = 8 \quad r(2) = 8$$

Pas d'élection !

$$r(4) = 3 \quad r(3) = 2$$



$$r(1) = 8 \quad r(2) = 7$$

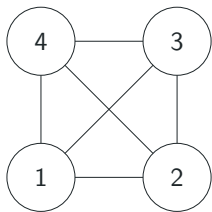
Élection

- Quelle est la probabilité de succès de l'algorithme (pour  $N = n$ ) ?

$$\mathbb{P}[\text{succes}] = n \times \sum_{i=2}^n \frac{1}{n} \cdot \left( \frac{i-1}{n} \right)^{n-1} \sim \frac{e^{-1}}{1 - e^{-1}} \approx 0,58$$

# ÉLECTION D'UN LEADER DANS UN RÉSEAU

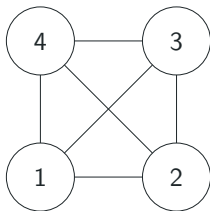
$$r(4) = 3 \quad r(3) = 2$$



$$r(1) = 8 \quad r(2) = 8$$

Pas d'élection !

$$r(4) = 3 \quad r(3) = 2$$



$$r(1) = 8 \quad r(2) = 7$$

Élection

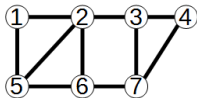
- Quelle est la probabilité de succès de l'algorithme (pour  $N = n$ ) ?

$$\mathbb{P}[\text{succes}] = n \times \sum_{i=2}^n \frac{1}{n} \cdot \left( \frac{i-1}{n} \right)^{n-1} \sim \frac{e^{-1}}{1 - e^{-1}} \approx 0,58$$

- En augmentant  $N$ , la probabilité tend vers 1

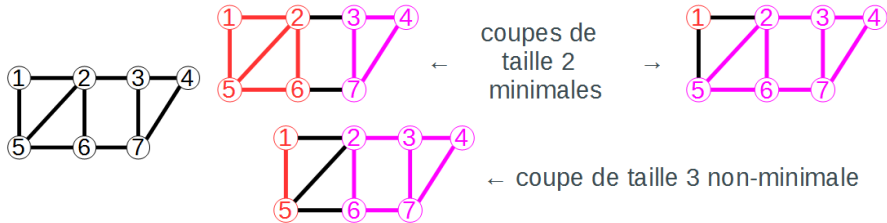
# ALGORITHME DE KARGER ET COUPE MINIMALE

- Considérons un graphe  $G = (V, E)$  à  $n$  sommets.
- Une coupe du graphe  $G$  est une partition des sommets en deux sous-ensembles  $V = V_1 \cup V_2$  ( $V_1 \cap V_2 = \emptyset$ )
- La taille d'une coupe  $V = V_1 \cup V_2$  est le nombre d'arêtes entre un sommet de  $V_1$  et un sommet de  $V_2$ .
- Ex : donnez une coupe minimale (de plus petite taille) du graphe suivant :



# ALGORITHME DE KARGER ET COUPE MINIMALE

- Considérons un graphe  $G = (V, E)$  à  $n$  sommets.
- Une coupe du graphe  $G$  est une partition des sommets en deux sous-ensembles  $V = V_1 \cup V_2$  ( $V_1 \cap V_2 = \emptyset$ )
- La taille d'une coupe  $V = V_1 \cup V_2$  est le nombre d'arêtes entre un sommet de  $V_1$  et un sommet de  $V_2$ .
- Ex : donnez une coupe minimale (de plus petite taille) du graphe suivant :



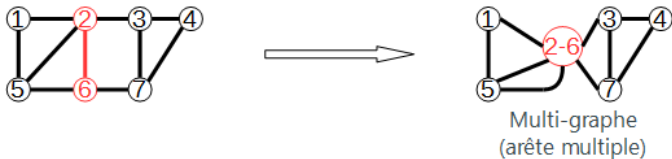
# ALGORITHME DE KARGER ET COUPE MINIMALE

**Problème MIN-CUT** : étant donné un graphe  $G = (V, E)$ , trouver une coupe minimale de  $G$ .

# ALGORITHME DE KARGER ET COUPE MINIMALE

**Problème MIN-CUT** : étant donné un graphe  $G = (V, E)$ , trouver une coupe minimale de  $G$ .

■ Contraction d'une arête (fusion de deux sommets) :

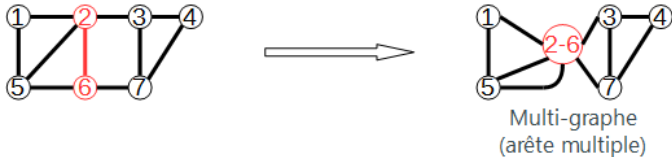




# ALGORITHME DE KARGER ET COUPE MINIMALE

**Problème MIN-CUT** : étant donné un graphe  $G = (V, E)$ , trouver une coupe minimale de  $G$ .

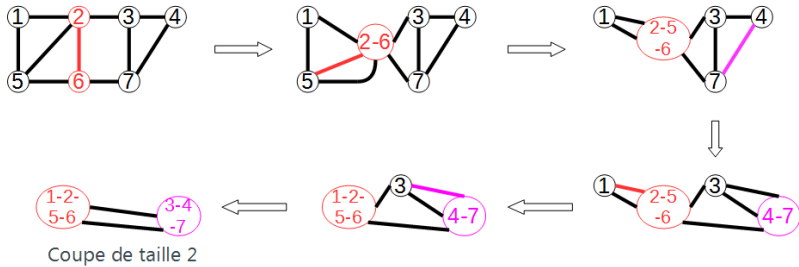
■ Contraction d'une arête (fusion de deux sommets) :



■ Algorithme de Karger

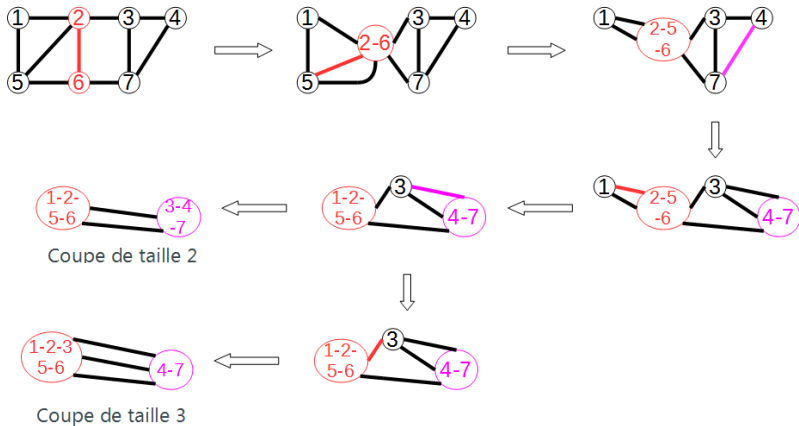
- ☐ Tant qu'une contraction est possible, en choisir une au hasard et contracter
- ☐ A la fin, il ne reste que deux sommets dont les labels forment une coupe
- ☐ Le résultat peut être erroné (algorithme de type Monte-Carlo)

# ALGORITHME DE KARGER ET COUPE MINIMALE





# ALGORITHME DE KARGER ET COUPE MINIMALE



- L'algorithme échoue/réussit à trouver une coupe minimale avec une certaine probabilité
- On ne sait pas lorsque l'algorithme échoue !

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .
- La probabilité de choisir une arête de la coupe est  $t/(tn/2) = 2/n$ .

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .
- La probabilité de choisir une arête de la coupe est  $t/(tn/2) = 2/n$ .
- La probabilité de ne pas choisir une arête de la coupe est donc  $1 - \frac{2}{n}$ .



# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .
- La probabilité de choisir une arête de la coupe est  $t/(tn/2) = 2/n$ .
- La probabilité de ne pas choisir une arête de la coupe est donc  $1 - \frac{2}{n}$ .
- Après la contraction d'une arête, le (multi-)graphe a  $n - 1$  sommets et une coupe minimale de taille  $t$  (si aucune arête de la coupe n'a été choisie)

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .
- La probabilité de choisir une arête de la coupe est  $t/(tn/2) = 2/n$ .
- La probabilité de ne pas choisir une arête de la coupe est donc  $1 - \frac{2}{n}$ .
- Après la contraction d'une arête, le (multi-)graphe a  $n - 1$  sommets et une coupe minimale de taille  $t$  (si aucune arête de la coupe n'a été choisie)
- Par récurrence sur  $n$ , la probabilité de ne jamais choisir une arête de la coupe minimale est

$$\geq \left(1 - \frac{2}{n}\right) \left(1 - \frac{2}{n-1}\right) \left(1 - \frac{2}{n-2}\right) \cdots \left(1 - \frac{2}{3}\right) = \frac{2}{n(n-1)} \geq \frac{2}{n^2}$$

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .
- La probabilité de choisir une arête de la coupe est  $t/(tn/2) = 2/n$ .
- La probabilité de ne pas choisir une arête de la coupe est donc  $1 - \frac{2}{n}$ .
- Après la contraction d'une arête, le (multi-)graphe a  $n - 1$  sommets et une coupe minimale de taille  $t$  (si aucune arête de la coupe n'a été choisie)
- Par récurrence sur  $n$ , la probabilité de ne jamais choisir une arête de la coupe minimale est

$$\geq \left(1 - \frac{2}{n}\right) \left(1 - \frac{2}{n-1}\right) \left(1 - \frac{2}{n-2}\right) \cdots \left(1 - \frac{2}{3}\right) = \frac{2}{n(n-1)} \geq \frac{2}{n^2}$$

- En exécutant  $cn^2/2$  fois l'algorithme de Karger, on trouve une coupe minimale avec une probabilité

# ALGORITHME DE KARGER ET COUPE MINIMALE

Supposons que le (multi-)graphe a  $n$  sommets et une coupe minimale de taille  $t$ .

- Le nombre d'arêtes du (multi-)graphe est  $\geq tn/2$ .
- La probabilité de choisir une arête de la coupe est  $t/(tn/2) = 2/n$ .
- La probabilité de ne pas choisir une arête de la coupe est donc  $1 - \frac{2}{n}$ .
- Après la contraction d'une arête, le (multi-)graphe a  $n - 1$  sommets et une coupe minimale de taille  $t$  (si aucune arête de la coupe n'a été choisie)
- Par récurrence sur  $n$ , la probabilité de ne jamais choisir une arête de la coupe minimale est

$$\geq \left(1 - \frac{2}{n}\right) \left(1 - \frac{2}{n-1}\right) \left(1 - \frac{2}{n-2}\right) \cdots \left(1 - \frac{2}{3}\right) = \frac{2}{n(n-1)} \geq \frac{2}{n^2}$$

- En exécutant  $cn^2/2$  fois l'algorithme de Karger, on trouve une coupe minimale avec une probabilité

$$1 - \left(1 - \frac{2}{n^2}\right)^{cn^2/2} \sim 1 - e^{-c} \xrightarrow{c \rightarrow \infty} 1$$

# ALGORITHMES DE MONTÉ-CARLO ET BIAIS

1. Un algorithme de Monté-Carlo est **biaisé vers le faux** si lorsqu'il répond *faux*, sa réponse est toujours correcte.
2. Un algorithme de Monté-Carlo est **biaisé vers le vrai** si lorsqu'il répond *vrai*, sa réponse est toujours correcte.

Exemples :

- Test de nullité d'un polynôme
- Test de primalité de Miller-Rabin

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines



# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines

**Algorithme 3 :** Tester 1 valeur (complexité :  $O(d)$ ) mais la probabilité d'échec est bornée par

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines

**Algorithme 3 :** Tester 1 valeur (complexité :  $O(d)$ ) mais la probabilité d'échec est bornée par  $d/q$

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines

**Algorithme 3 :** Tester 1 valeur (complexité :  $O(d)$ ) mais la probabilité d'échec est bornée par  $d/q$

**Algorithme 4 :** Tester  $k$  valeurs aléatoires (complexité :  $O(kd)$ ) mais la probabilité d'échec est bornée par

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines

**Algorithme 3 :** Tester 1 valeur (complexité :  $O(d)$ ) mais la probabilité d'échec est bornée par  $d/q$

**Algorithme 4 :** Tester  $k$  valeurs aléatoires (complexité :  $O(kd)$ ) mais la probabilité d'échec est bornée par  $(d/q)^k$

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines

**Algorithme 3 :** Tester 1 valeur (complexité :  $O(d)$ ) mais la probabilité d'échec est bornée par  $d/q$

**Algorithme 4 :** Tester  $k$  valeurs aléatoires (complexité :  $O(kd)$ ) mais la probabilité d'échec est bornée par  $(d/q)^k$

→ Si  $q \gg d$ , alors l'algorithme a peu de chance de se tromper.

# TEST DE NULLITÉ D'UN POLYNÔME

Soit  $P(X)$  un polynôme de degré  $d$  sur un corps (fini ou infini) très grand.

**Problème :** La fonction  $x \rightarrow P(x)$  est-elle nulle ?

Exemple : la fonction  $x \mapsto x^q - x$  est nulle sur le corps fini  $\mathbb{F}_q$ .

**Algorithme 1 :** Tester toutes les valeurs (complexité :  $O(qd)$ )

**Algorithme 2 :** Tester  $d + 1$  valeurs (complexité :  $O(d^2)$ ) car un polynôme de degré  $d$  a au plus  $d$  racines

**Algorithme 3 :** Tester 1 valeur (complexité :  $O(d)$ ) mais la probabilité d'échec est bornée par  $d/q$

**Algorithme 4 :** Tester  $k$  valeurs aléatoires (complexité :  $O(kd)$ ) mais la probabilité d'échec est bornée par  $(d/q)^k$

→ Si  $q \gg d$ , alors l'algorithme a peu de chance de se tromper.

Rem : le résultat est le même pour des polynômes multivariés !

# TEST DE NULLITÉ D'UN POLYNÔME

## Test polynôme nul

**Entrée :**  $P \in \mathbb{F}[x_1, \dots, x_n]$  un polynôme sur le corps  $\mathbb{F}$

$S \subset \mathbb{F}$  un sous-ensemble fini de  $\mathbb{F}$

**Sortie :** Retourne si  $x \mapsto P(x)$  est nulle

1 : Choisir  $(v_1, \dots, v_n)$  uniformément dans  $S$

2 : Calculer  $r = P(v_1, \dots, v_n)$

3 : **Si**  $r \neq 0$  **Alors**

4 :     **Retourner** *non – nul*

5 : **Sinon**

6 :     **Retourner** *probablement nul*

7 : **Fin Si**

■ C'est un algorithme de Monté-Carlo biaisé vers le *faux* (non-nul)

■ La probabilité d'erreur dans l'autre cas est  $\leq \frac{d}{\text{Card}(S)}$

■ En effectuant au plus  $k$  appels au test, la probabilité d'erreur  $\leq \left( \frac{d}{\text{Card}(S)} \right)^k$

# TEST DE NULLITÉ D'UN POLYNÔME

Exemple :

■ Posons  $P = x_1^2 - x_1 \cdot x_2^2$  sur le corps fini  $\mathbb{F}_7 = \mathbb{Z}/7\mathbb{Z}$

■ Considérons  $S = F$ . Nous avons  $d = 3$  et  $\text{Card}(S) = 7$  soit,

$$\text{si } (v_1, v_2) \in_{\mathcal{R}} S \times S, \quad \Pr [P(v_1, v_2) = 0] \leq \frac{d}{\text{Card}(S)} = \frac{3}{7}$$

■ Vérification :

| $x_1 \backslash x_2$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|----------------------|---|---|---|---|---|---|---|
| 0                    | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1                    | 1 | 0 | 4 | 6 | 6 | 4 | 0 |
| 2                    | 4 | 2 | 3 | 0 | 0 | 3 | 2 |
| 3                    | 2 | 6 | 4 | 3 | 3 | 4 | 6 |
| 4                    | 2 | 5 | 0 | 1 | 1 | 0 | 5 |
| 5                    | 4 | 6 | 5 | 1 | 1 | 5 | 6 |
| 6                    | 1 | 2 | 5 | 3 | 3 | 5 | 2 |

$$\text{si } (v_1, v_2) \in_{\mathcal{R}} S \times S, \quad \Pr [P(v_1, v_2) = 0] = \frac{13}{49} \leq \frac{3}{7}$$



- $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, \dots\}$
- `http://www.lix.polytechnique.fr/~morain/Primes/myprimes.html`
- `http://primes.utm.edu/`

■ Il existe une infinité de nombres premiers

■  $\pi(x) = \#\{n \leq x : n \text{ est premier}\}$

On a

$$\pi(x) \sim x / \ln x$$

■ Si on tire uniformément un nombre  $\leq x$ , la probabilité qu'il soit premier est  $1 / \ln x$

- Il existe une infinité de nombres premiers

- $\pi(x) = \#\{n \leq x : n \text{ est premier}\}$

On a

$$\pi(x) \sim x / \ln x$$

- Si on tire uniformément un nombre  $\leq x$ , la probabilité qu'il soit premier est  $1 / \ln x$

Deux problèmes :

- tester la primalité (Monté-Carlo)
- construire des nombres premiers de taille fixée (Las Vegas)

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors

$$a^{p-1} = 1 \pmod{p}$$

■ Test de primalité d'un entier  $N$  :

Choisir  $a \leq N$  aléatoirement et calculer  $s = 2^{N-1} \pmod{N}$

$$N \text{ premier} \implies s = 1$$

$$s \neq 1 \implies N \text{ non premier}$$

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors

$$a^{p-1} = 1 \pmod{p}$$

■ Test de primalité d'un entier  $N$  :

Choisir  $a \leq N$  aléatoirement et calculer  $s = 2^{N-1} \pmod{N}$

$$N \text{ premier} \implies s = 1$$

$$s \neq 1 \implies N \text{ non premier}$$

Condition **nécessaire** de primalité.

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors

$$a^{p-1} = 1 \pmod{p}$$

■ Test de primalité d'un entier  $N$  :

Choisir  $a \leq N$  aléatoirement et calculer  $s = 2^{N-1} \pmod{N}$

$$N \text{ premier} \implies s = 1$$

$$s \neq 1 \implies N \text{ non premier}$$

Condition **nécessaire** de primalité.

$$N = 341, a = 2 \rightsquigarrow 2^{340} = 1 \pmod{341}$$

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors

$$a^{p-1} = 1 \pmod{p}$$

■ Test de primalité d'un entier  $N$  :

Choisir  $a \leq N$  aléatoirement et calculer  $s = 2^{N-1} \pmod{N}$

$$N \text{ premier} \implies s = 1$$

$$s \neq 1 \implies N \text{ non premier}$$

Condition **nécessaire** de primalité.

$$N = 341, a = 2 \rightsquigarrow 2^{340} = 1 \pmod{341} \quad \text{mais} \quad 341 = 13 \times 11$$

Il existe des nombres entiers composés  $N$ , appelés *nombres de Carmichael* qui vérifient :

$$\forall a \in \mathbb{N}, \text{pgcd}(a, N) = 1 : a^{N-1} = 1 \pmod{N}$$

Autrement dit, ils passent toujours le précédent test de primalité!!!

$$\begin{aligned} 561 &= 3 \times 11 \times 17 \\ 1105 &= 5 \times 13 \times 17 \\ 1729 &= 7 \times 13 \times 19 \\ 2465 &= 5 \times 17 \times 29 \\ 2821 &= 7 \times 13 \times 31 \\ 6601 &= 7 \times 23 \times 41 \\ 8911 &= 7 \times 19 \times 67 \end{aligned}$$



## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

■ Soit  $N$  un entier impair premier.

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair

$$\text{Ex :} \quad N = 101 \quad N - 1 = 100 = 2^2 \cdot 25$$

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair  
Ex :  $N = 101$       $N - 1 = 100 = 2^2 \cdot 25$
- En appliquant l'identité remarquable  $(x - 1)(x + 1) = x^2 - 1$  plusieurs fois, nous avons

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair

$$\text{Ex : } \quad N = 101 \quad N - 1 = 100 = 2^2 \cdot 25$$

- En appliquant l'identité remarquable  $(x - 1)(x + 1) = x^2 - 1$  plusieurs fois, nous avons

$$a^{N-1} - 1 = a^{2^s \cdot u} - 1 = (a^u - 1)(a^u + 1)(a^{2u} + 1)(a^{4u} + 1) \dots (a^{2^{s-1}u} + 1)$$

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair

$$\text{Ex : } \quad N = 101 \quad N - 1 = 100 = 2^2 \cdot 25$$

- En appliquant l'identité remarquable  $(x - 1)(x + 1) = x^2 - 1$  plusieurs fois, nous avons

$$a^{N-1} - 1 = a^{2^s \cdot u} - 1 = (a^u - 1)(a^u + 1)(a^{2u} + 1)(a^{4u} + 1) \dots (a^{2^{s-1}u} + 1)$$

$$\text{Ex : } \quad N = 101 \quad a^{N-1} - 1 = a^{100} - 1 = (a^{25} - 1)(a^{25} + 1)(a^{50} - 1)$$

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair

$$\text{Ex : } \quad N = 101 \quad N - 1 = 100 = 2^2 \cdot 25$$

- En appliquant l'identité remarquable  $(x - 1)(x + 1) = x^2 - 1$  plusieurs fois, nous avons

$$a^{N-1} - 1 = a^{2^s \cdot u} - 1 = (a^u - 1)(a^u + 1)(a^{2u} + 1)(a^{4u} + 1) \dots (a^{2^{s-1}u} + 1)$$

$$\text{Ex : } \quad N = 101 \quad a^{N-1} - 1 = a^{100} - 1 = (a^{25} - 1)(a^{25} + 1)(a^{50} - 1)$$

- Comme  $N$  est premier, le produit vaut  $0 \pmod{N}$  selon Fermat

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair

$$\text{Ex : } \quad N = 101 \quad N - 1 = 100 = 2^2 \cdot 25$$

- En appliquant l'identité remarquable  $(x - 1)(x + 1) = x^2 - 1$  plusieurs fois, nous avons

$$a^{N-1} - 1 = a^{2^s \cdot u} - 1 = (a^u - 1)(a^u + 1)(a^{2u} + 1)(a^{4u} + 1) \dots (a^{2^{s-1}u} + 1)$$

$$\text{Ex : } \quad N = 101 \quad a^{N-1} - 1 = a^{100} - 1 = (a^{25} - 1)(a^{25} + 1)(a^{50} - 1)$$

- Comme  $N$  est premier, le produit vaut  $0 \pmod{N}$  selon Fermat
- Autrement dit, l'une des parenthèses est nulle modulo  $N$  !!!

## THÉORÈME DE FERMAT

Soit  $p$  un entier **premier** et  $a \in \mathbb{Z}$  tel que  $\text{pgcd}(a, p) = 1$ . Alors  $a^{p-1} = 1 \pmod{p}$

- Soit  $N$  un entier impair premier.
- On peut écrire de manière unique  $N - 1 = 2^s \cdot u$  avec  $u$  impair

$$\text{Ex : } \quad N = 101 \quad N - 1 = 100 = 2^2 \cdot 25$$

- En appliquant l'identité remarquable  $(x - 1)(x + 1) = x^2 - 1$  plusieurs fois, nous avons

$$a^{N-1} - 1 = a^{2^s \cdot u} - 1 = (a^u - 1)(a^u + 1)(a^{2u} + 1)(a^{4u} + 1) \dots (a^{2^{s-1}u} + 1)$$

$$\text{Ex : } \quad N = 101 \quad a^{N-1} - 1 = a^{100} - 1 = (a^{25} - 1)(a^{25} + 1)(a^{50} - 1)$$

- Comme  $N$  est premier, le produit vaut  $0 \pmod{N}$  selon Fermat
- Autrement dit, l'une des parenthèses est nulle modulo  $N$  !!!

$$\text{Ex : } \quad N = 101, \quad a = 2, \quad 2^{50} + 1 = 0 \pmod{101}$$



# TEST DE PRIMALITÉ DE MILLER-RABIN

## Test de primalité de Miller-Rabin

**Entrée :**  $N$  un entier impair

**Sortie :** Retourne *non – premier* ou *probablement – premier*

1 : Calculer  $s$  et  $u$  tels que  $N - 1 = 2^s \cdot u$  avec  $u$  impair

2 : Choisir  $a$  premier avec  $N$  aléatoirement dans l'intervalle  $]1, N[$

3 : **Vérifier** si  $a^u - 1 \equiv 0 \pmod{N}$  ou  $a^u + 1 \equiv 0 \pmod{N}$  ou  
 $a^{2^u} + 1 \equiv 0 \pmod{N}$  ou ... ou  $a^{2^{s-1}u} + 1 \equiv 0 \pmod{N}$

4 : **Si** l'une des égalités est vraie **Alors**

5 :     **Retourner** *probablement – premier*

6 : **Sinon**

7 :     **Retourner** *non – premier*

8 : **Fin Si**

# TEST DE PRIMALITÉ DE MILLER-RABIN

## Test de primalité de Miller-Rabin

**Entrée :**  $N$  un entier impair

**Sortie :** Retourne *non – premier* ou *probablement – premier*

1 : Calculer  $s$  et  $u$  tels que  $N - 1 = 2^s \cdot u$  avec  $u$  impair

2 : Choisir  $a$  premier avec  $N$  aléatoirement dans l'intervalle  $]1, N[$

3 : **Vérifier** si  $a^u - 1 \equiv 0 \pmod{N}$  ou  $a^u + 1 \equiv 0 \pmod{N}$  ou  
 $a^{2^s u} + 1 \equiv 0 \pmod{N}$  ou ... ou  $a^{2^{s-1} u} + 1 \equiv 0 \pmod{N}$

4 : **Si** l'une des égalités est vraie **Alors**

5 :     **Retourner** *probablement – premier*

6 : **Sinon**

7 :     **Retourner** *non – premier*

8 : **Fin Si**

■ C'est un algorithme de Monté-Carlo biaisé vers le *faux* (non-premier)

# TEST DE PRIMALITÉ DE MILLER-RABIN

## Test de primalité de Miller-Rabin

**Entrée :**  $N$  un entier impair

**Sortie :** Retourne *non – premier* ou *probablement – premier*

1 : Calculer  $s$  et  $u$  tels que  $N - 1 = 2^s \cdot u$  avec  $u$  impair

2 : Choisir  $a$  premier avec  $N$  aléatoirement dans l'intervalle  $]1, N[$

3 : **Vérifier** si  $a^u - 1 \equiv 0 \pmod{N}$  ou  $a^u + 1 \equiv 0 \pmod{N}$  ou  
 $a^{2^s} + 1 \equiv 0 \pmod{N}$  ou ... ou  $a^{2^{s-1}u} + 1 \equiv 0 \pmod{N}$

4 : **Si** l'une des égalités est vraie **Alors**

5 :     **Retourner** *probablement – premier*

6 : **Sinon**

7 :     **Retourner** *non – premier*

8 : **Fin Si**

■ C'est un algorithme de Monté-Carlo biaisé vers le *faux* (non-premier)

■ La probabilité d'erreur dans l'autre cas est  $p_1 \leq \frac{1}{4}$

# TEST DE PRIMALITÉ DE MILLER-RABIN

## Test de primalité de Miller-Rabin

**Entrée :**  $N$  un entier impair

**Sortie :** Retourne *non – premier* ou *probablement – premier*

1 : Calculer  $s$  et  $u$  tels que  $N - 1 = 2^s \cdot u$  avec  $u$  impair

2 : Choisir  $a$  premier avec  $N$  aléatoirement dans l'intervalle  $]1, N[$

3 : **Vérifier si**  $a^u - 1 \equiv 0 \pmod{N}$  ou  $a^u + 1 \equiv 0 \pmod{N}$  ou  
 $a^{2^s u} + 1 \equiv 0 \pmod{N}$  ou ... ou  $a^{2^{s-1} u} + 1 \equiv 0 \pmod{N}$

4 : **Si** l'une des égalités est vraie **Alors**

5 :     **Retourner** *probablement – premier*

6 : **Sinon**

7 :     **Retourner** *non – premier*

8 : **Fin Si**

- C'est un algorithme de Monté-Carlo biaisé vers le *faux* (non-premier)
- La probabilité d'erreur dans l'autre cas est  $p_1 \leq \frac{1}{4}$
- En effectuant au plus  $k$  appels au test, la probabilité d'erreur  $p_k \leq 4^{-k}$

## ALGORITHME DE TYPE LAS VEGAS

Un algorithme de type Las Vegas est un **algorithme probabiliste**

- dont le résultat est toujours correct,
- mais dont l'espérance du temps de calcul est bornée :
  - pour toute entrée  $x$ , on pose  $T(x)$  la variable aléatoire égale au temps de calcul sur  $x$ .
  - Alors il existe une fonction positive  $f : \mathbb{N} \rightarrow \mathbb{R}_+$  telle que

$$\mathbb{E}[T(x)] \leq f(|x|)$$

avec  $|x|$  la taille de l'entrée  $x$  (nombre de bits en mémoire)

# ALGORITHMES DE TYPE LAS VEGAS

## Exemple 1 :

Considérons un algorithme qui émet un 1 avec probabilité  $1/3$  et un 0 avec probabilité  $2/3$ . L'algorithme réitère son expérience jusqu'à émettre un 1.

*Quel est l'espérance du temps de calcul ?*

# ALGORITHMES DE TYPE LAS VEGAS

## Exemple 1 :

Considérons un algorithme qui émet un 1 avec probabilité  $1/3$  et un 0 avec probabilité  $2/3$ . L'algorithme réitère son expérience jusqu'à émettre un 1.

*Quel est l'espérance du temps de calcul ?*

→ le nombre de tirages aléatoires suit loi géométrique de paramètre  $p = 1/3$ .  
L'espérance du nombre de tirages est donc  $\frac{1}{p} = 3$ .

## Exemple 1 :

Considérons un algorithme qui émet un 1 avec probabilité  $1/3$  et un 0 avec probabilité  $2/3$ . L'algorithme réitère son expérience jusqu'à émettre un 1.

*Quel est l'espérance du temps de calcul ?*

→ le nombre de tirages aléatoires suit loi géométrique de paramètre  $p = 1/3$ .  
L'espérance du nombre de tirages est donc  $\frac{1}{p} = 3$ .

## Exemple 2 :

Génération de nombres premiers ....



# PARADOXE DES ANNIVERSAIRES

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

# PARADOXE DES ANNIVERSAIRES

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $365^k$  valeurs possibles.

# PARADOXE DES ANNIVERSAIRES

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $365^k$  valeurs possibles.
- Parmi ces valeurs,  $365(365 - 1)(365 - 2) \cdots (365 - k + 1)$  ont des  $x_i$  tous différents.

# PARADOXE DES ANNIVERSAIRES

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $365^k$  valeurs possibles.
- Parmi ces valeurs,  $365(365 - 1)(365 - 2) \cdots (365 - k + 1)$  ont des  $x_i$  tous différents.
- La probabilité que  $k$  élèves sont nés des jours tous différents est

$$p_k = \frac{365(365 - 1)(365 - 2) \cdots (365 - k + 1)}{365^k} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{365}\right)$$

# PARADOXE DES ANNIVERSAIRES

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $365^k$  valeurs possibles.
- Parmi ces valeurs,  $365(365 - 1)(365 - 2) \cdots (365 - k + 1)$  ont des  $x_i$  tous différents.
- La probabilité que  $k$  élèves sont nés des jours tous différents est

$$p_k = \frac{365(365 - 1)(365 - 2) \cdots (365 - k + 1)}{365^k} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{365}\right)$$

- On a  $p_{21} = 0.52$  et  $p_{22} = 0.49$ .

# PARADOXE DES ANNIVERSAIRES

**Question :** Combien d'élèves faut-il dans une classe pour avoir plus d'une chance sur 2 que deux élèves soient nés le même jour ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $365^k$  valeurs possibles.
- Parmi ces valeurs,  $365(365 - 1)(365 - 2) \cdots (365 - k + 1)$  ont des  $x_i$  tous différents.
- La probabilité que  $k$  élèves sont nés des jours tous différents est

$$p_k = \frac{365(365 - 1)(365 - 2) \cdots (365 - k + 1)}{365^k} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{365}\right)$$

- On a  $p_{21} = 0.52$  et  $p_{22} = 0.49$ .
- A partir de 22 élèves, il y a plus d'une chance sur 2 que deux élèves soient nés le même jour !

# PARADOXE DES ANNIVERSAIRES : CAS GÉNÉRAL

**Question :** Soit  $E$  un ensemble de cardinal  $n$ . Soit  $(x_1, \dots, x_k) \in E^k$  des éléments choisis aléatoirement et uniformément dans  $E$ . Pour quelle valeur de  $k$  a-t-on **plus d'une chance sur 12** que deux  $x_i$  soient égaux ?

**Réponse :**

# PARADOXE DES ANNIVERSAIRES : CAS GÉNÉRAL

**Question :** Soit  $E$  un ensemble de cardinal  $n$ . Soit  $(x_1, \dots, x_k) \in E^k$  des éléments choisis aléatoirement et uniformément dans  $E$ . Pour quelle valeur de  $k$  a-t-on **plus d'une chance sur 12** que deux  $x_i$  soient égaux ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $n^k$  valeurs possibles.



# PARADOXE DES ANNIVERSAIRES : CAS GÉNÉRAL

**Question :** Soit  $E$  un ensemble de cardinal  $n$ . Soit  $(x_1, \dots, x_k) \in E^k$  des éléments choisis aléatoirement et uniformément dans  $E$ . Pour quelle valeur de  $k$  a-t-on **plus d'une chance sur 12** que deux  $x_i$  soient égaux ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $n^k$  valeurs possibles.
- Parmi ces valeurs, seules  $n(n-1)(n-2) \cdots (n-k+1)$  ont des  $x_i$  différents 2 à 2.

# PARADOXE DES ANNIVERSAIRES : CAS GÉNÉRAL

**Question :** Soit  $E$  un ensemble de cardinal  $n$ . Soit  $(x_1, \dots, x_k) \in E^k$  des éléments choisis aléatoirement et uniformément dans  $E$ . Pour quelle valeur de  $k$  a-t-on **plus d'une chance sur 12** que deux  $x_i$  soient égaux ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $n^k$  valeurs possibles.
- Parmi ces valeurs, seules  $n(n-1)(n-2) \cdots (n-k+1)$  ont des  $x_i$  différents 2 à 2.
- La probabilité que les  $k$   $x_i$  sont différents est

$$p_k = \frac{n(n-1)(n-2) \cdots (n-k+1)}{n^k} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{n}\right)$$

$$p_k \leq \exp\left(\frac{-1}{n} \sum_{i=1}^{k-1} i\right) = \exp\left(\frac{-k(k-1)}{2n}\right).$$

# PARADOXE DES ANNIVERSAIRES : CAS GÉNÉRAL

**Question :** Soit  $E$  un ensemble de cardinal  $n$ . Soit  $(x_1, \dots, x_k) \in E^k$  des éléments choisis aléatoirement et uniformément dans  $E$ . Pour quelle valeur de  $k$  a-t-on **plus d'une chance sur 12** que deux  $x_i$  soient égaux ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $n^k$  valeurs possibles.
- Parmi ces valeurs, seules  $n(n-1)(n-2) \cdots (n-k+1)$  ont des  $x_i$  différents 2 à 2.
- La probabilité que les  $k$   $x_i$  sont différents est

$$p_k = \frac{n(n-1)(n-2) \cdots (n-k+1)}{n^k} = \prod_{i=1}^{k-1} \left(1 - \frac{i}{n}\right)$$

$$p_k \leq \exp\left(\frac{-1}{n} \sum_{i=1}^{k-1} i\right) = \exp\left(\frac{-k(k-1)}{2n}\right).$$

- si  $k \sim \sqrt{2n \ln 12}$  alors  $p_k \approx 1/12$

# PARADOXE DES ANNIVERSAIRES : CAS GÉNÉRAL

**Question :** Soit  $E$  un ensemble de cardinal  $n$ . Soit  $(x_1, \dots, x_k) \in E^k$  des éléments choisis aléatoirement et uniformément dans  $E$ . Pour quelle valeur de  $k$  a-t-on **plus d'une chance sur 12** que deux  $x_i$  soient égaux ?

**Réponse :**

- Le  $k$ -uplet  $(x_1, \dots, x_k)$  peut prendre  $n^k$  valeurs possibles.
- Parmi ces valeurs, seules  $n(n-1)(n-2) \cdots (n-k+1)$  ont des  $x_i$  différents 2 à 2.
- La probabilité que les  $k$   $x_i$  sont différents est  $p_k$
- si  $k \sim \sqrt{2n \ln 12}$  alors  $p_k \approx 1/12$

**Conclusion :** Il suffit que  $k$  soit de l'ordre de  $\sqrt{n}$  pour que la probabilité de ne pas avoir une collision soit plus petite que  $1/2$ .

# PRÉSENTATION DU PROBLÈME

Soit  $E$  un ensemble de cardinal  $n$ .

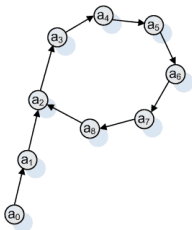
Soit  $f : E \rightarrow E$  une fonction.

Soit  $a_0$  un élément de  $E$ . On pose

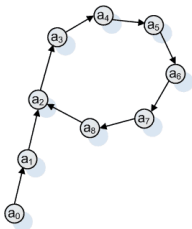
$$a_{i+1} = f(a_i).$$

## PROBLÈME

La suite  $(a_i)_{i \in \mathbb{N}}$  est ultimement périodique de période  $\ell_c$ . Déterminer la valeur  $\ell_c$  de la période (aussi appelée cycle).

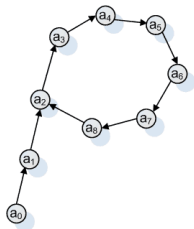


# PRÉSENTATION DU PROBLÈME



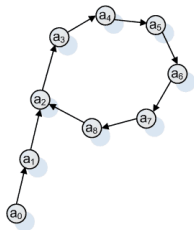
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$

# PRÉSENTATION DU PROBLÈME



■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3,

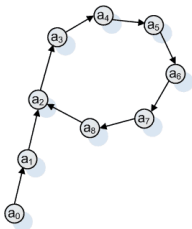
# PRÉSENTATION DU PROBLÈME



■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10,

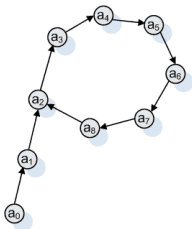


# PRÉSENTATION DU PROBLÈME



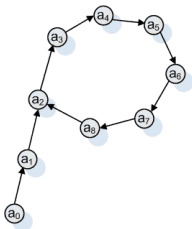
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1,

# PRÉSENTATION DU PROBLÈME



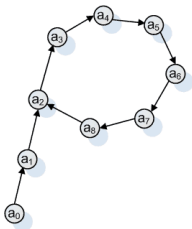
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2,

# PRÉSENTATION DU PROBLÈME



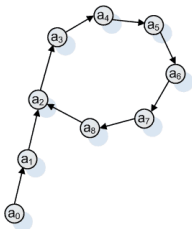
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5,

# PRÉSENTATION DU PROBLÈME



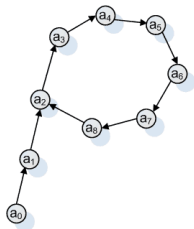
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6,

# PRÉSENTATION DU PROBLÈME



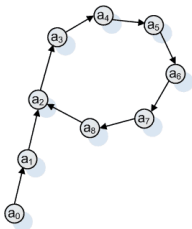
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20,

# PRÉSENTATION DU PROBLÈME



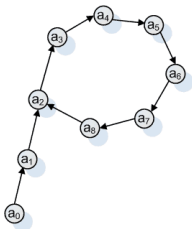
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17,

# PRÉSENTATION DU PROBLÈME



■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10,

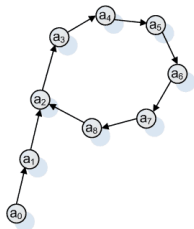
# PRÉSENTATION DU PROBLÈME



- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$

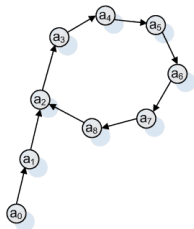


# PRÉSENTATION DU PROBLÈME



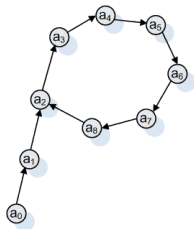
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32,

# PRÉSENTATION DU PROBLÈME



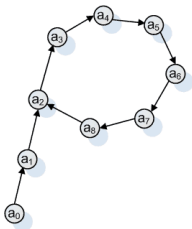
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504,

# PRÉSENTATION DU PROBLÈME



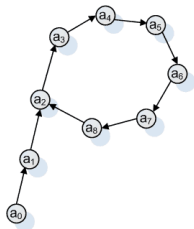
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290,

# PRÉSENTATION DU PROBLÈME



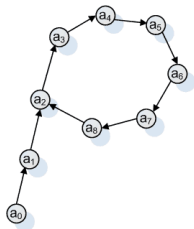
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220,

# PRÉSENTATION DU PROBLÈME



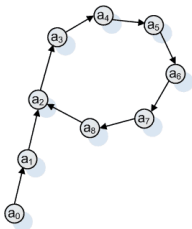
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469,

# PRÉSENTATION DU PROBLÈME



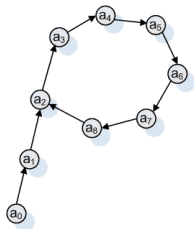
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100

# PRÉSENTATION DU PROBLÈME



- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102

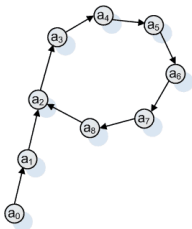
# PRÉSENTATION DU PROBLÈME



- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102, 506

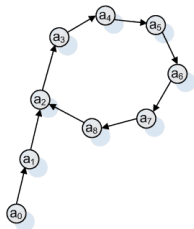


# PRÉSENTATION DU PROBLÈME



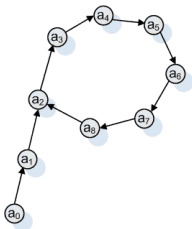
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102, 506, 226

# PRÉSENTATION DU PROBLÈME



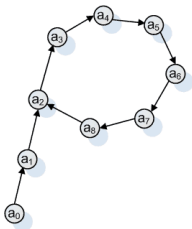
- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102, 506, 226, 19

# PRÉSENTATION DU PROBLÈME



- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102, 506, 226, 19, 362

# PRÉSENTATION DU PROBLÈME



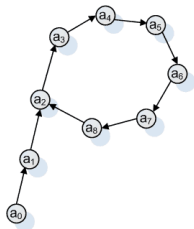
■ Exemple :  $f(x) = x^2 + 1 \pmod{20}$

3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...

■ Exemple :  $f(x) = x^2 + 1 \pmod{521}$

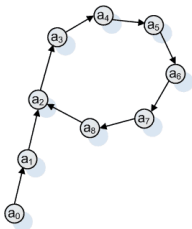
32, 504, 290, 220, 469, 100, 102, 506, 226, 19, 362, 274, 53,  
205, 346, 408, 266, 422, 424, 32, 504, ...

# PRÉSENTATION DU PROBLÈME



- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102, 506, 226, 19, 362, 274, 53,  
205, 346, 408, 266, 422, 424, 32, 504, ...
- Remarque : entre les deux 32  $\rightsquigarrow$  18 éléments

# PRÉSENTATION DU PROBLÈME



- Exemple :  $f(x) = x^2 + 1 \pmod{20}$   
3, 10, 1, 2, 5, 6, 20, 17, 10, 1, 2, ...
- Exemple :  $f(x) = x^2 + 1 \pmod{521}$   
32, 504, 290, 220, 469, 100, 102, 506, 226, 19, 362, 274, 53,  
205, 346, 408, 266, 422, 424, 32, 504, ...
- Remarque : entre les deux 32  $\rightsquigarrow$  18 éléments  
 $\sqrt{521} \simeq 22,8$ .

# ALGORITHME NAÏF

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur du cycle de la suite  
 $(a_i)_{i \in \mathbb{N}}$

$S = \{\}$

$a = a_0$

$i = 0$

Stocker  $(a, 0)$  dans l'ensemble  $S$

**Tant que true faire**

$a = f(a)$

$i = i + 1$

**Si** il existe  $(a, j)$  dans  $S$  **alors**

        retourner  $i - j$

**sinon**

        Ajouter  $(a, i)$  à  $S$ .

**fin si**

**fin Tant que**

# ALGORITHME NAÏF

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$S = \{\}$

$a = a_0$

$i = 0$

Stocker  $(a, 0)$  dans l'ensemble  $S$

**Tant que true faire**

$a = f(a)$

$i = i + 1$

**Si** il existe  $(a, j)$  dans  $S$  **alors**

        retourner  $i - j$

**sinon**

        Ajouter  $(a, i)$  à  $S$ .

**fin si**

**fin Tant que**

Complexité :

■ temps :  $O(\text{card}(E))$  dans le pire des cas



# ALGORITHME NAÏF

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$S = \{\}$

$a = a_0$

$i = 0$

Stocker  $(a, 0)$  dans l'ensemble  $S$

**Tant que true faire**

$a = f(a)$

$i = i + 1$

**Si** il existe  $(a, j)$  dans  $S$  **alors**

        retourner  $i - j$

**sinon**

        Ajouter  $(a, i)$  à  $S$ .

**fin si**

**fin Tant que**

Complexité :

- temps :  $O(\text{card}(E))$  dans le pire des cas
- mémoire :  $O(\text{card}(E))$  dans le pire des cas

# ALGORITHME NAÏF

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$S = \{\}$

$a = a_0$

$i = 0$

Stocker  $(a, 0)$  dans l'ensemble  $S$

**Tant que true faire**

$a = f(a)$

$i = i + 1$

**Si** il existe  $(a, j)$  dans  $S$  **alors**  
        retourner  $i - j$

**sinon**

        Ajouter  $(a, i)$  à  $S$ .

**fin si**

**fin Tant que**

Complexité :

- temps :  $O(\text{card}(E))$  dans le pire des cas
- mémoire :  $O(\text{card}(E))$  dans le pire des cas
- temps moyen :  $O(\sqrt{\text{card}(E)})$  (paradoxe des anniversaires)

# ALGORITHME NAÏF

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$S = \{\}$

$a = a_0$

$i = 0$

Stocker  $(a, 0)$  dans l'ensemble  $S$

**Tant que true faire**

$a = f(a)$

$i = i + 1$

**Si** il existe  $(a, j)$  dans  $S$  **alors**  
        retourner  $i - j$

**sinon**

        Ajouter  $(a, i)$  à  $S$ .

**fin si**

**fin Tant que**

Complexité :

- temps :  $O(\text{card}(E))$  dans le pire des cas
- mémoire :  $O(\text{card}(E))$  dans le pire des cas
- temps moyen :  $O(\sqrt{\text{card}(E)})$  (paradoxe des anniversaires)
- espace moyen :  $O(\sqrt{\text{card}(E)})$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$a_0 = 3$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$a_0 = 3 \quad a_1 = 10$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$a_0 = 3 \quad a_1 = 10 \quad a_2 = 0$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$a_0 = 3 \quad a_1 = 10 \quad a_2 = 0 \quad a_3 = 1$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$a_0 = 3 \quad a_1 = 10 \quad a_2 = 0 \quad a_3 = 1 \quad a_4 = 2$$



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$a_0 = 3 \quad a_1 = 10 \quad a_2 = 0 \quad a_3 = 1 \quad a_4 = 2 \quad a_5 = 5$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{cccccc} a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\ a_6 = 26 & & & & & \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{cccccc} a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\ a_6 = 26 & a_7 = 71 & & & & \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{cccccc} a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\ a_6 = 26 & a_7 = 71 & a_8 = 93 & & & \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{cccccc} a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\ a_6 = 26 & a_7 = 71 & a_8 = 93 & a_9 = 65 & & \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{cccccc} a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\ a_6 = 26 & a_7 = 71 & a_8 = 93 & a_9 = 65 & a_{10} = 85 & \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|            |            |            |            |               |               |
|------------|------------|------------|------------|---------------|---------------|
| $a_0 = 3$  | $a_1 = 10$ | $a_2 = 0$  | $a_3 = 1$  | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$ | $a_7 = 71$ | $a_8 = 93$ | $a_9 = 65$ | $a_{10} = 85$ | $a_{11} = 55$ |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{llllll} a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\ a_6 = 26 & a_7 = 71 & a_8 = 93 & a_9 = 65 & a_{10} = 85 & a_{11} = 55 \\ a_{12} = 97 \end{array}$$



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |            |            |               |               |
|---------------|---------------|------------|------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$  | $a_3 = 1$  | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$ | $a_9 = 65$ | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ |            |            |               |               |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

$$\begin{array}{llllll}
 a_0 = 3 & a_1 = 10 & a_2 = 0 & a_3 = 1 & a_4 = 2 & a_5 = 5 \\
 a_6 = 26 & a_7 = 71 & a_8 = 93 & a_9 = 65 & a_{10} = 85 & a_{11} = 55 \\
 a_{12} = 97 & a_{13} = 17 & a_{14} = 88 & & & 
 \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ |               |               |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ | $a_{16} = 15$ |               |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ | $a_{16} = 15$ | $a_{17} = 24$ |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ | $a_{16} = 15$ | $a_{17} = 24$ |
| $a_{18} = 72$ |               |               |               |               |               |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ | $a_{16} = 15$ | $a_{17} = 24$ |
| $a_{18} = 72$ | $a_{19} = 34$ |               |               |               |               |

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ | $a_{16} = 15$ | $a_{17} = 24$ |
| $a_{18} = 72$ | $a_{19} = 34$ | $a_{20} = 46$ |               |               |               |



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1$ ,  $a_0 = 3$

Algorithme naïf :

|               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|
| $a_0 = 3$     | $a_1 = 10$    | $a_2 = 0$     | $a_3 = 1$     | $a_4 = 2$     | $a_5 = 5$     |
| $a_6 = 26$    | $a_7 = 71$    | $a_8 = 93$    | $a_9 = 65$    | $a_{10} = 85$ | $a_{11} = 55$ |
| $a_{12} = 97$ | $a_{13} = 17$ | $a_{14} = 88$ | $a_{15} = 69$ | $a_{16} = 15$ | $a_{17} = 24$ |
| $a_{18} = 72$ | $a_{19} = 34$ | $a_{20} = 46$ | $a_{21} = 97$ |               |               |

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  
 $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la  
suite  $(a_i)_{i \in \mathbb{N}}$

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

Principes :

■ à la sortie de la première boucle,  
 $a_m = a_{2m}$

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

## Principes :

- à la sortie de la première boucle,  
 $a_m = a_{2m}$
- soit  $2m - m = m$  est un multiple de la longueur du cycle

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

Principes :

- à la sortie de la première boucle,  
 $a_m = a_{2m}$
- soit  $2m - m = m$  est un multiple de la longueur du cycle
- après la deuxième boucle,  
 $a_{m_1} = a_{2m_1}$  et  $m_1 \leq m + \mu$

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

Principes :

- à la sortie de la première boucle,  
 $a_m = a_{2m}$
- soit  $2m - m = m$  est un multiple de la longueur du cycle
- après la deuxième boucle,  
 $a_{m_1} = a_{2m_1}$  et  $m_1 \leq m + \mu$
- $2m_1 - m_1 = m_1$  est aussi un multiple de  $\mu$  soit  $\mu = m_1 - m$ .

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

Principes :

- à la sortie de la première boucle,  
 $a_m = a_{2m}$
- soit  $2m - m = m$  est un multiple de la longueur du cycle
- après la deuxième boucle,  
 $a_{m_1} = a_{2m_1}$  et  $m_1 \leq m + \mu$
- $2m_1 - m_1 = m_1$  est aussi un multiple de  $\mu$  soit  $\mu = m_1 - m$ .
- temps moyen :  $O(\sqrt{\text{card}(E)})$   
(paradoxe des anniversaires)

# ALGORITHME DE DÉTECTION DE CYCLE DE FLOYD

**Entrée :** la fonction  $f : E \rightarrow E$  et  $a_0 \in E$

**Sortie :** La longueur  $\mu$  du cycle de la suite  $(a_i)_{i \in \mathbb{N}}$

$m = 0; \quad x = y = a_0$

**Faire**

$m = m + 1$

$x = f(x) \quad (x \leftarrow a_m)$

$y = f(f(y)) \quad (y \leftarrow a_{2m})$

**Tant que**  $x \neq y$

$m_1 = m$

**Faire**

$m_1 = m_1 + 1$

$x = f(x) \quad (x \leftarrow a_{m_1})$

$y = f(f(y)) \quad (y \leftarrow a_{2m_1})$

**Tant que**  $x \neq y$

**Retourner**  $\mu = m_1 - m$

**Principes :**

- à la sortie de la première boucle,  
 $a_m = a_{2m}$
- soit  $2m - m = m$  est un multiple de la longueur du cycle
- après la deuxième boucle,  
 $a_{m_1} = a_{2m_1}$  et  $m_1 \leq m + \mu$
- $2m_1 - m_1 = m_1$  est aussi un multiple de  $\mu$  soit  $\mu = m_1 - m$ .
- temps moyen :  $O(\sqrt{\text{card}(E)})$   
(paradoxe des anniversaires)
- espace moyen  $O(1)$



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

Algorithme de Floyd :

Phase 1 :

$$\begin{array}{cccccc} \left\{ \begin{array}{l} a_0 = 3 \\ a_0 = 3 \end{array} \right. & \left\{ \begin{array}{l} a_1 = 10 \\ a_2 = 0 \end{array} \right. & \left\{ \begin{array}{l} a_2 = 0 \\ a_4 = 2 \end{array} \right. & \left\{ \begin{array}{l} a_3 = 1 \\ a_6 = 26 \end{array} \right. & \left\{ \begin{array}{l} a_4 = 2 \\ a_8 = 93 \end{array} \right. & \left\{ \begin{array}{l} a_5 = 5 \\ a_{10} = 85 \end{array} \right. \end{array}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases}$$



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases} \quad \begin{cases} a_{13} = 17 \\ a_{26} = 24 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases} \quad \begin{cases} a_{13} = 17 \\ a_{26} = 24 \end{cases} \quad \begin{cases} a_{14} = 88 \\ a_{28} = 34 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases} \quad \begin{cases} a_{13} = 17 \\ a_{26} = 24 \end{cases} \quad \begin{cases} a_{14} = 88 \\ a_{28} = 34 \end{cases} \quad \begin{cases} a_{15} = 69 \\ a_{30} = 97 \end{cases}$$



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases} \quad \begin{cases} a_{13} = 17 \\ a_{26} = 24 \end{cases} \quad \begin{cases} a_{14} = 88 \\ a_{28} = 34 \end{cases} \quad \begin{cases} a_{15} = 69 \\ a_{30} = 97 \end{cases} \quad \begin{cases} a_{16} = 15 \\ a_{32} = 88 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases} \quad \begin{cases} a_{13} = 17 \\ a_{26} = 24 \end{cases} \quad \begin{cases} a_{14} = 88 \\ a_{28} = 34 \end{cases} \quad \begin{cases} a_{15} = 69 \\ a_{30} = 97 \end{cases} \quad \begin{cases} a_{16} = 15 \\ a_{32} = 88 \end{cases} \quad \begin{cases} a_{17} = 24 \\ a_{34} = 15 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_0 = 3 \\ a_0 = 3 \end{cases} \quad \begin{cases} a_1 = 10 \\ a_2 = 0 \end{cases} \quad \begin{cases} a_2 = 0 \\ a_4 = 2 \end{cases} \quad \begin{cases} a_3 = 1 \\ a_6 = 26 \end{cases} \quad \begin{cases} a_4 = 2 \\ a_8 = 93 \end{cases} \quad \begin{cases} a_5 = 5 \\ a_{10} = 85 \end{cases}$$

$$\begin{cases} a_6 = 26 \\ a_{12} = 97 \end{cases} \quad \begin{cases} a_7 = 71 \\ a_{14} = 88 \end{cases} \quad \begin{cases} a_8 = 93 \\ a_{16} = 15 \end{cases} \quad \begin{cases} a_9 = 65 \\ a_{18} = 72 \end{cases} \quad \begin{cases} a_{10} = 85 \\ a_{20} = 46 \end{cases} \quad \begin{cases} a_{11} = 55 \\ a_{22} = 17 \end{cases}$$

$$\begin{cases} a_{12} = 97 \\ a_{24} = 69 \end{cases} \quad \begin{cases} a_{13} = 17 \\ a_{26} = 24 \end{cases} \quad \begin{cases} a_{14} = 88 \\ a_{28} = 34 \end{cases} \quad \begin{cases} a_{15} = 69 \\ a_{30} = 97 \end{cases} \quad \begin{cases} a_{16} = 15 \\ a_{32} = 88 \end{cases} \quad \begin{cases} a_{17} = 24 \\ a_{34} = 15 \end{cases}$$

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \quad \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \quad \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \quad \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases}$$



# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \quad \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \quad \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \quad \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \quad \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \quad \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \quad \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \quad \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \quad \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \quad \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \quad \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \quad \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

$$\begin{cases} a_{25} = 15 \\ a_{50} = 88 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \quad \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \quad \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \quad \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \quad \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \quad \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

$$\begin{cases} a_{25} = 15 \\ a_{50} = 88 \end{cases} \quad \begin{cases} a_{26} = 24 \\ a_{52} = 15 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

$$\begin{cases} a_{25} = 15 \\ a_{50} = 88 \end{cases} \begin{cases} a_{26} = 24 \\ a_{52} = 15 \end{cases} \begin{cases} a_{27} = 72 \\ a_{54} = 72 \end{cases}$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

$$\begin{cases} a_{25} = 15 \\ a_{50} = 88 \end{cases} \begin{cases} a_{26} = 24 \\ a_{52} = 15 \end{cases} \begin{cases} a_{27} = 72 \\ a_{54} = 72 \end{cases} \Rightarrow m_1 = 27$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

$$\begin{cases} a_{25} = 15 \\ a_{50} = 88 \end{cases} \begin{cases} a_{26} = 24 \\ a_{52} = 15 \end{cases} \begin{cases} a_{27} = 72 \\ a_{54} = 72 \end{cases} \Rightarrow m_1 = 27 \Rightarrow \mu = m_1 - m = 9$$

# EXEMPLE D'EXÉCUTION

On pose  $E = \{0, 1, 2, \dots, 100\}$ ,  $f(x) = x^2 + 1 \pmod{100}$ ,  $a_0 = 3$

## Algorithme de Floyd :

Phase 1 :

$$\begin{cases} a_{18} = 72 \\ a_{36} = 72 \end{cases} \Rightarrow m = 18$$

Phase 2 :

$$\begin{cases} a_{19} = 34 \\ a_{38} = 46 \end{cases} \begin{cases} a_{20} = 46 \\ a_{40} = 17 \end{cases} \begin{cases} a_{21} = 97 \\ a_{42} = 69 \end{cases} \begin{cases} a_{22} = 17 \\ a_{44} = 24 \end{cases} \begin{cases} a_{23} = 88 \\ a_{46} = 34 \end{cases} \begin{cases} a_{24} = 69 \\ a_{48} = 97 \end{cases}$$

$$\begin{cases} a_{25} = 15 \\ a_{50} = 88 \end{cases} \begin{cases} a_{26} = 24 \\ a_{52} = 15 \end{cases} \begin{cases} a_{27} = 72 \\ a_{54} = 72 \end{cases} \Rightarrow m_1 = 27 \Rightarrow \mu = m_1 - m = 9$$

L'algorithme de Floyd est utilisé pour résoudre les problèmes de **factorisation** et de **logarithme discret** en cryptographie.



## Que sait-on sur les nombres premiers ?

- Il y a une infinité de nombres premiers
- La proportion de nombres premiers est de plus en plus faible

$$\text{Nb de nombres premiers} \leq n : \quad \pi(n) \sim \frac{n}{\ln n}$$

$$\text{Proportion de nombres premiers} \leq n : \quad \frac{\pi(n)}{n} \sim \frac{1}{\ln n}$$

- Entre deux nombres premiers, l'intervalle peut être aussi grand que l'on veut

**Problème** : comment générer un nombre premier d'au plus  $n$  bits ?

# GÉNÉRATION DE NOMBRES PREMIERS

## Que sait-on sur les nombres premiers ?

- Il y a une infinité de nombres premiers
- La proportion de nombres premiers est de plus en plus faible

$$\text{Nb de nombres premiers} \leq n : \quad \pi(n) \sim \frac{n}{\ln n}$$

$$\text{Proportion de nombres premiers} \leq n : \quad \frac{\pi(n)}{n} \sim \frac{1}{\ln n}$$

- Entre deux nombres premiers, l'intervalle peut être aussi grand que l'on veut

**Problème** : comment générer un nombre premier d'au plus  $n$  bits ?

Gen\_Prime( $n$ ) :

```
p = uniforme(1, 2n)
tant que p n'est pas premier
    p = uniforme(1, 2n)
retourner p
```

# LOI GÉOMÉTRIQUE ET GÉNÉRATION DE NOMBRES PREMIERS

Proportion de nombres premiers  $< 2^n$  :  $\frac{\pi(2^n)}{2^n} \sim \frac{1}{n \ln 2}$

```
Gen_Prime(n) :  
  p = uniforme(1,2^n)  
  tant que p n'est pas premier  
    p = uniforme(1,2^n)  
  retourner p
```

# LOI GÉOMÉTRIQUE ET GÉNÉRATION DE NOMBRES PREMIERS

Proportion de nombres premiers  $< 2^n$  :  $\frac{\pi(2^n)}{2^n} \sim \frac{1}{n \ln 2}$

```
Gen_Prime(n) :  
  p = uniforme(1,2^n)  
  tant que p n'est pas premier  
    p = uniforme(1,2^n)  
  retourner p
```

C'est un algorithme de type Las Vegas.

La probabilité de trouver est un nombre premier à chaque étape est  $p \sim 1/n \ln 2$ .

# LOI GÉOMÉTRIQUE ET GÉNÉRATION DE NOMBRES PREMIERS

Proportion de nombres premiers  $< 2^n$  :  $\frac{\pi(2^n)}{2^n} \sim \frac{1}{n \ln 2}$

```

Gen_Prime(n) :
  p = uniforme(1, 2^n)
  tant que p n'est pas premier
    p = uniforme(1, 2^n)
  retourner p
    
```

C'est un algorithme de type Las Vegas.

La probabilité de trouver est un nombre premier à chaque étape est  $p \sim 1/n \ln 2$ .

Espérance et variance du nombre d'itérations :

$$\text{espérance} = \frac{1}{p} \sim n \ln 2, \quad \text{variance} = \frac{1-p}{p^2} \sim (n \ln 2)^2$$

L'algorithme effectue en moyenne un nombre d'itérations linéaire en le nombre de bits (très efficace !)

## Processus :

- Soit  $G_n$  un graphe à  $n$  sommets sans arête.
- Il possède au départ  $n$  composantes connexes.
- On ajoute les arêtes une à une **aléatoirement et uniformément**.

**Question** : combien d'arêtes faut-il en moyenne pour obtenir un graphe connexe ?

# LOI GÉOMÉTRIQUE ET TEST DE CONNEXITÉ

## Processus :

- Soit  $G_n$  un graphe à  $n$  sommets sans arête.
- Il possède au départ  $n$  composantes connexes.
- On ajoute les arêtes une à une **aléatoirement et uniformément**.

**Question** : combien d'arêtes faut-il en moyenne pour obtenir un graphe connexe ?

## Notations :

- Soit  $X_k$  : v.a. du nombre d'arêtes ajoutées pour passer de  $k$  à  $k - 1$  CC
- $X = \sum_{k=2}^n X_k$  nombre total d'arêtes ajoutées
- A chaque arête ajoutée  $(i, j)$ , nous avons deux possibilités :
  1. si  $i$  et  $j$  appartiennent à la même composante alors on ne fait rien
  2. si  $i$  et  $j$  appartiennent à deux composantes différentes, on fusionne ces composantes (le nombre de composantes diminue).
- Soit  $p_k$  la probabilité d'être dans le cas 2. (on passe de  $k$  à  $k - 1$  CC).

# LOI GÉOMÉTRIQUE ET TEST DE CONNEXITÉ

## Notations :

- Soit  $X_k$  : v.a. du nombre d'arêtes ajoutées pour passer de  $k$  à  $k - 1$  CC
- $X = \sum_{k=2}^n X_k$  nombre total d'arêtes ajoutées
- A chaque arête ajouté  $(i, j)$ , nous avons deux possibilités :
  1. si  $i$  et  $j$  appartiennent à la même composante alors on ne fait rien
  2. si  $i$  et  $j$  appartiennent à deux composantes différentes, on fusionne ces composantes (le nombre de composantes diminue).
- Soit  $p_k$  la probabilité d'être dans le cas 2. (on passe de  $k$  à  $k - 1$  CC).

Pendant la phase  $k$ ,  $X_k$  suit une loi géométrique de paramètre  $p_k$  (algorithme Las Vegas).

$$E[X_k] = \frac{1}{p_k}, \quad V(X_k) = \frac{1 - p_k}{p_k^2}$$



# LOI GÉOMÉTRIQUE ET TEST DE CONNEXITÉ

## Notations :

- Soit  $p_k$  la probabilité d'être dans le cas 2. (on passe de  $k$  à  $k - 1$  CC).

Pendant la phase  $k$ ,  $X_k$  suit une loi géométrique de paramètre  $p_k$  (algorithme Las Vegas).

$$E[X_k] = \frac{1}{p_k}, \quad V(X_k) = \frac{1 - p_k}{p_k^2}$$

## Minoration de $p_k$

Soit  $C_i$  la composante de  $i$ , chaque CC  $C$  possède au moins un sommet  $j$ , donc cela fait au moins  $k - 1$  sommets  $j$  possibles pour être dans le cas 2. D'où

$$p_k \geq \frac{k - 1}{n - 1}.$$

# LOI GÉOMÉTRIQUE ET TEST DE CONNEXITÉ

Minoration de  $p_k$

$$p_k \geq \frac{k-1}{n-1}.$$

Espérance

$$E[X] = \sum_{k=2}^n E[X_k] \leq (n-1) \sum_{k=2}^n \frac{1}{k-1} = (n-1)H_{n-1},$$

où  $H_{n-1}$  est le  $n-1$ ème nombre harmonique. On montre que

$$H_n = \ln n + \gamma + \frac{1}{2n} + O\left(\frac{1}{n^2}\right),$$

où  $\gamma$  est une constante (la constante d'Euler). Donc  $H_n \sim \ln n$ . Il faut en moyenne ajouter  $n \ln n$  arêtes (en fait moins car nous calculons une majoration) pour obtenir un graphe connexe.

## Variance

$$\text{Var}(X) = \sum_{k=2}^n \frac{1 - p_k}{p_k^2} \leq \sum_{k=2}^n \frac{(n - k)(n - 1)}{(k - 1)^2}.$$

$$\text{Var}(X) \leq \frac{\pi^2}{6} n^2 - n \ln n.$$

# LOI GÉOMÉTRIQUE ET TEST DE CONNEXITÉ

## Variance

$$\text{Var}(X) = \sum_{k=2}^n \frac{1 - p_k}{p_k^2} \leq \sum_{k=2}^n \frac{(n - k)(n - 1)}{(k - 1)^2}.$$

$$\text{Var}(X) \leq \frac{\pi^2}{6} n^2 - n \ln n.$$

En appliquant l'inégalité de Bienaymé-Chebychev

$$\Pr(|X - E[X]| \geq \frac{\pi n}{\sqrt{6}} t) \leq \frac{1}{t^2},$$

avec  $t = \ln \ln n$ , il vient

$$\Pr(X \geq n \ln n + n \ln \ln n) = O\left(\frac{1}{(\ln \ln n)^2}\right).$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Un enfant collectionne des coupons en achetant des paquets de céréales. On suppose qu'il existe  $n$  coupons à collectionner et que chaque coupon a la même probabilité d'apparaître dans un paquet.

## PROBLÈME DU COLLECTIONNEUR DE COUPONS

Combien de paquets l'enfant doit-il acheter en moyenne pour avoir au moins un exemplaire de chacun des coupons ?

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Un enfant collectionne des coupons en achetant des paquets de céréales. On suppose qu'il existe  $n$  coupons à collectionner et que chaque coupon a la même probabilité d'apparaître dans un paquet.

## PROBLÈME DU COLLECTIONNEUR DE COUPONS

Combien de paquets l'enfant doit-il acheter en moyenne pour avoir au moins un exemplaire de chacun des coupons ?

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q1-** Exprimez  $X$  en fonction des  $X_i$ .

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Un enfant collectionne des coupons en achetant des paquets de céréales. On suppose qu'il existe  $n$  coupons à collectionner et que chaque coupon a la même probabilité d'apparaître dans un paquet.

## PROBLÈME DU COLLECTIONNEUR DE COUPONS

Combien de paquets l'enfant doit-il acheter en moyenne pour avoir au moins un exemplaire de chacun des coupons ?

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q1-** Exprimez  $X$  en fonction des  $X_i$ .

$$X = X_1 + \dots + X_n$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q2-** A un moment donné, l'enfant possède  $i - 1$  coupons. Quelle est la probabilité  $p_i$  qu'il trouve un nouveau coupon dans un paquet ?



# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q2-** A un moment donné, l'enfant possède  $i - 1$  coupons. Quelle est la probabilité  $p_i$  qu'il trouve un nouveau coupon dans un paquet ?

L'enfant trouve un nouveau coupon s'il obtient un coupon parmi les  $n - i$  coupons qu'il ne possède pas, comme la distribution est uniforme, on obtient

$$p_i = \frac{n - i + 1}{n}.$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q2-** A un moment donné, l'enfant possède  $i - 1$  coupons. Quelle est la probabilité  $p_i$  qu'il trouve un nouveau coupon dans un paquet ?

L'enfant trouve un nouveau coupon s'il obtient un coupon parmi les  $n - i$  coupons qu'il ne possède pas, comme la distribution est uniforme, on obtient

$$p_i = \frac{n - i + 1}{n}.$$

**Q3-** Quelle est la loi de la variable aléatoire  $X_i$  ? (Donner espérance et variance)

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q2-** A un moment donné, l'enfant possède  $i - 1$  coupons. Quelle est la probabilité  $p_i$  qu'il trouve un nouveau coupon dans un paquet ?

L'enfant trouve un nouveau coupon s'il obtient un coupon parmi les  $n - i$  coupons qu'il ne possède pas, comme la distribution est uniforme, on obtient

$$p_i = \frac{n - i + 1}{n}.$$

**Q3-** Quelle est la loi de la variable aléatoire  $X_i$  ? (Donner espérance et variance)

C'est une loi géométrique de paramètre  $p_i$ .  $\mathbb{E}[X_i] = \frac{1}{p_i}$ ,  $\mathbb{V}(X_i) = \frac{1-p_i}{p_i^2}$ .

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q2-** A un moment donné, l'enfant possède  $i - 1$  coupons. Quelle est la probabilité  $p_i$  qu'il trouve un nouveau coupon dans un paquet ?

L'enfant trouve un nouveau coupon s'il obtient un coupon parmi les  $n - i$  coupons qu'il ne possède pas, comme la distribution est uniforme, on obtient

$$p_i = \frac{n - i + 1}{n}.$$

**Q3-** Quelle est la loi de la variable aléatoire  $X_i$  ? (Donner espérance et variance)

C'est une loi géométrique de paramètre  $p_i$ .  $\mathbb{E}[X_i] = \frac{1}{p_i}$ ,  $\mathbb{V}(X_i) = \frac{1-p_i}{p_i^2}$ .

**Q4-** Rappelez la valeur de la somme  $\sum_{i=1}^n \frac{1}{i}$ .

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q2-** A un moment donné, l'enfant possède  $i - 1$  coupons. Quelle est la probabilité  $p_i$  qu'il trouve un nouveau coupon dans un paquet ?

L'enfant trouve un nouveau coupon s'il obtient un coupon parmi les  $n - i$  coupons qu'il ne possède pas, comme la distribution est uniforme, on obtient

$$p_i = \frac{n - i + 1}{n}.$$

**Q3-** Quelle est la loi de la variable aléatoire  $X_i$  ? (Donner espérance et variance)

C'est une loi géométrique de paramètre  $p_i$ .  $\mathbb{E}[X_i] = \frac{1}{p_i}$ ,  $\mathbb{V}(X_i) = \frac{1-p_i}{p_i^2}$ .

**Q4-** Rappelez la valeur de la somme  $\sum_{i=1}^n \frac{1}{i}$ .

$$\sum_{i=1}^n \frac{1}{i} = H_n = \ln n + \gamma + \frac{1}{2n} + O\left(\frac{1}{n^2}\right), \text{ le } n^{\text{ième}} \text{ nombre harmonique.}$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q5-** Calculez l'espérance de  $X$  et sa variance.

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q5-** Calculez l'espérance de  $X$  et sa variance.

$$\begin{aligned}\mathbb{E}[X] &= \sum_{i=1}^n \frac{1}{p_i} \\ &= \sum_{i=1}^n \frac{n}{n-i+1} = n \sum_{i=1}^n \frac{1}{i} \\ &= nH_n \sim n \ln n.\end{aligned}$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

Soit  $X_i$  le nombre de paquets achetés pour passer de  $i - 1$  coupons à  $i$  coupons et  $X$  le nombre total de paquets achetés.

**Q5-** Calculez l'espérance de  $X$  et sa variance.

$$\begin{aligned}\mathbb{E}[X] &= \sum_{i=1}^n \frac{1}{p_i} \\ &= \sum_{i=1}^n \frac{n}{n-i+1} = n \sum_{i=1}^n \frac{1}{i} \\ &= nH_n \sim n \ln n.\end{aligned}$$

$$\begin{aligned}\mathbb{V}(X) &= \sum_{i=1}^n \mathbb{V}(X_i) = \sum_{i=1}^n \frac{1-p_i}{p_i^2} \\ &= \sum_{i=1}^n \frac{1}{p_i^2} - \sum_{i=1}^n \frac{1}{p_i} \\ &= n^2 \sum_{i=1}^n \frac{1}{i^2} - nH_n \\ &\sim \frac{n^2 \pi^2}{6} - n \ln n \sim \frac{n^2 \pi^2}{6}\end{aligned}$$



# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q6-** Supposons  $n = 30$ . Combien doit-il acheter de paquets pour avoir moins d'une chance sur 100 de ne pas avoir tous les coupons ?

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q6-** Supposons  $n = 30$ . Combien doit-il acheter de paquets pour avoir moins d'une chance sur 100 de ne pas avoir tous les coupons ?

- En moyenne, on doit collectionner  $30 \cdot H_{30} \approx 102$  coupons pour obtenir la collection complète des 30 sortes de coupons.

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q6-** Supposons  $n = 30$ . Combien doit-il acheter de paquets pour avoir moins d'une chance sur 100 de ne pas avoir tous les coupons ?

- En moyenne, on doit collectionner  $30 \cdot H_{30} \approx 102$  coupons pour obtenir la collection complète des 30 sortes de coupons.
- La variance vaut  $\mathbb{V}(X) \sim \frac{n^2 \pi^2}{6} - n \ln n \approx 1376$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q6-** Supposons  $n = 30$ . Combien doit-il acheter de paquets pour avoir moins d'une chance sur 100 de ne pas avoir tous les coupons ?

- En moyenne, on doit collectionner  $30 \cdot H_{30} \approx 102$  coupons pour obtenir la collection complète des 30 sortes de coupons.
- La variance vaut  $\mathbb{V}(X) \sim \frac{n^2 \pi^2}{6} - n \ln n \approx 1376$
- Selon l'inégalité de Bienaymé-Chebychev

$$\mathbb{P} (|X - \mathbb{E}[X]| \geq t) \leq \frac{\mathbb{V}(X)}{t^2}$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q6-** Supposons  $n = 30$ . Combien doit-il acheter de paquets pour avoir moins d'une chance sur 100 de ne pas avoir tous les coupons ?

- En moyenne, on doit collectionner  $30 \cdot H_{30} \approx 102$  coupons pour obtenir la collection complète des 30 sortes de coupons.
- La variance vaut  $\mathbb{V}(X) \sim \frac{n^2 \pi^2}{6} - n \ln n \approx 1376$
- Selon l'inégalité de Bienaymé-Chebychev

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq t) \leq \frac{\mathbb{V}(X)}{t^2}$$

- En prenant  $t = \sqrt{100\mathbb{V}(X)}$  on obtient

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq t) \leq \frac{1}{100}$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q6-** Supposons  $n = 30$ . Combien doit-il acheter de paquets pour avoir moins d'une chance sur 100 de ne pas avoir tous les coupons ?

- En moyenne, on doit collectionner  $30 \cdot H_{30} \approx 102$  coupons pour obtenir la collection complète des 30 sortes de coupons.
- La variance vaut  $\mathbb{V}(X) \sim \frac{n^2 \pi^2}{6} - n \ln n \approx 1376$
- Selon l'inégalité de Bienaymé-Chebychev

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq t) \leq \frac{\mathbb{V}(X)}{t^2}$$

- En prenant  $t = \sqrt{100\mathbb{V}(X)}$  on obtient

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq t) \leq \frac{1}{100}$$

- soit

$$\mathbb{P}(X \geq \mathbb{E}[X] + t) = \mathbb{P}(X \geq 102 + 371) \leq \frac{1}{100}$$

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q7-** Comment utilisez ce problème pour tester un générateur pseudo-aléatoire ?

# LOI GÉOMÉTRIQUE ET COLLECTIONNEUR DE COUPONS

**Q7-** Comment utilisez ce problème pour tester un générateur pseudo-aléatoire ?

1. On découpe la suite aléatoire en blocs de taille  $m$  (par exemple,  $m = 10$ ).
2. On a alors une collection de  $n = 2^{10} = 1024$  coupons (chaque mot binaire de longueur  $m$  est un coupon).
3. On regarde si le nombre de blocs nécessaire pour avoir la collection complète se rapproche de  $\mathbb{E}[X]$ .
4. On peut aussi le faire un grand nombre de fois et regarder la répartition.



# QUICKSORT ET RANDOMQUICKSORT

```
partition(tableau T, entier début, entier fin, entier pivot)
échanger T[pivot] et T[fin]
  j := début
  pour i de début à fin - 1
    si T[i] <= T[fin] alors
      échanger T[i] et T[j]
      j := j + 1
  échanger T[fin] et T[j]
renvoyer j
```

```
Quicksort(tableau T, entier début, entier fin)
  si début < fin alors
    pivot := début
    pivot := partition(T, début, fin, pivot)
    Quicksort(T, début, pivot-1)
    Quicksort(T, pivot+1, fin)
  fin si

RandomQuicksort(tableau T, entier début, entier fin)
  si début < fin alors
    pivot := RandomPivot(T,début,fin)
    pivot := partition(T, début, fin, pivot)
    RandomQuicksort(T, début, pivot-1)
    RandomQuicksort(T, pivot+1, fin)
  fin si
```

# REMARQUES SUR QUICKSORT

- C'est un algorithme déterministe
- Pire des cas : quand le tableau est trié !
- Complexité dans le pire de cas :  $O(n^2)$  comparaisons
- Dans le modèle des permutations aléatoires, la complexité moyenne de Quicksort vérifie

$$C_n = n - 1 + \frac{1}{n} \sum_{i=0}^{n-1} (C_i + C_{n-i-1}) \sim 2n \ln n.$$

En moyenne, QuickSort est **quasi-linéaire** !

# REMARQUES SUR QUICKSORT

- C'est un algorithme déterministe
- Pire des cas : quand le tableau est trié !
- Complexité dans le pire de cas :  $O(n^2)$  comparaisons
- Dans le modèle des permutations aléatoires, la complexité moyenne de Quicksort vérifie

$$C_n = n - 1 + \frac{1}{n} \sum_{i=0}^{n-1} (C_i + C_{n-i-1}) \sim 2n \ln n.$$

En moyenne, QuickSort est **quasi-linéaire** !

Mais cette différence peut être un problème dans certaines applications, surtout celles proches du pire des cas ...

# ETUDE DE RANDOMQUICKSORT

Soit  $C$  la v.a. du nombre de comparaisons de RandomQuickSort.

Soient  $i$  et  $j \in \{1, \dots, n\}$  avec  $i < j$ . Soit  $C_{i,j}$  la v.a définie par

$$\begin{aligned} C_{i,j} &= 1 \text{ lorsque } i \text{ et } j \text{ sont comparés entre-eux} \\ &= 0 \text{ sinon} \end{aligned}$$

# ETUDE DE RANDOMQUICKSORT

Soit  $C$  la v.a. du nombre de comparaisons de RandomQuickSort.

Soient  $i$  et  $j \in \{1, \dots, n\}$  avec  $i < j$ . Soit  $C_{i,j}$  la v.a définie par

$$\begin{aligned} C_{i,j} &= 1 \text{ lorsque } i \text{ et } j \text{ sont comparés entre-eux} \\ &= 0 \text{ sinon} \end{aligned}$$

Il vient

$$C = \sum_{i,j \in \{1, \dots, n\}, i < j} C_{i,j}.$$

# ETUDE DE RANDOMQUICKSORT

Soit  $C$  la v.a. du nombre de comparaisons de RandomQuickSort.

Soient  $i$  et  $j \in \{1, \dots, n\}$  avec  $i < j$ . Soit  $C_{i,j}$  la v.a définie par

$$\begin{aligned} C_{i,j} &= 1 \text{ lorsque } i \text{ et } j \text{ sont comparés entre-eux} \\ &= 0 \text{ sinon} \end{aligned}$$

Il vient

$$C = \sum_{i,j \in \{1, \dots, n\}, i < j} C_{i,j}.$$

Notons

$$p_{i,j} = \mathbb{P}(C_{i,j} = 1) = \mathbb{E}[C_{i,j}].$$

Il vient

$$\mathbb{E}[C] = \sum_{i,j \in \{1, \dots, n\}, i < j} \mathbb{E}[C_{i,j}] = \sum_{i,j \in \{1, \dots, n\}, i < j} p_{i,j}.$$

Il reste donc à calculer  $p_{i,j}$  !!!

# ETUDE DE RANDOMQUICKSORT

Il reste donc à calculer  $p_{i,j}$  !!!

- Au début,  $i$  et  $j$  sont dans les mêmes parties du tableau et peuvent être comparés
- Si  $i$  ou  $j$  est choisi comme pivot, alors  $i$  et  $j$  sont comparés 1 fois, et ne seront plus comparés ensuite
- Si le pivot est choisi dans l'intervalle  $]i, j[$ , alors  $i$  et  $j$  ne seront plus dans la même partie du tableau et ne seront plus comparés

La probabilité du deuxième cas est donnée par

# ETUDE DE RANDOMQUICKSORT

Il reste donc à calculer  $p_{i,j}$  !!!

- Au début,  $i$  et  $j$  sont dans les mêmes parties du tableau et peuvent être comparés
- Si  $i$  ou  $j$  est choisi comme pivot, alors  $i$  et  $j$  sont comparés 1 fois, et ne seront plus comparés ensuite
- Si le pivot est choisi dans l'intervalle  $]i, j[$ , alors  $i$  et  $j$  ne seront plus dans la même partie du tableau et ne seront plus comparés

La probabilité du deuxième cas est donnée par  $p_{i,j} = \frac{2}{j - i + 1}$



# ETUDE DE RANDOMQUICKSORT

Il reste donc à calculer  $p_{i,j}$  !!!

- Au début,  $i$  et  $j$  sont dans les mêmes parties du tableau et peuvent être comparés
- Si  $i$  ou  $j$  est choisi comme pivot, alors  $i$  et  $j$  sont comparés 1 fois, et ne seront plus comparés ensuite
- Si le pivot est choisi dans l'intervalle  $]i, j[$ , alors  $i$  et  $j$  ne seront plus dans la même partie du tableau et ne seront plus comparés

La probabilité du deuxième cas est donnée par  $p_{i,j} = \frac{2}{j-i+1}$

$$\begin{aligned}\mathbb{E}[C] &= \sum_{(i,j) \in \{1 \leq i < j \leq n\}} \frac{2}{j-i+1} \\ &= \dots \\ &= 2nH_n - 2n + 3 \sim 2n \ln n\end{aligned}$$

→ C'est la même asymptotique que pour la complexité en moyenne de QuickSort !

# ETUDE DE RANDOMQUICKSORT

Il reste donc à calculer  $p_{i,j}$  !!!

- Au début,  $i$  et  $j$  sont dans les mêmes parties du tableau et peuvent être comparés
- Si  $i$  ou  $j$  est choisi comme pivot, alors  $i$  et  $j$  sont comparés 1 fois, et ne seront plus comparés ensuite
- Si le pivot est choisi dans l'intervalle  $]i, j[$ , alors  $i$  et  $j$  ne seront plus dans la même partie du tableau et ne seront plus comparés

La probabilité du deuxième cas est donnée par  $p_{i,j} = \frac{2}{j-i+1}$

$$\begin{aligned}\mathbb{E}[C] &= \sum_{(i,j) \in \{1 \leq i < j \leq n\}} \frac{2}{j-i+1} \\ &= \dots \\ &= 2nH_n - 2n + 3 \sim 2n \ln n\end{aligned}$$

→ C'est la même asymptotique que pour la complexité en moyenne de QuickSort !  
→ Mais la méthode garantit une complexité en moyenne quasi-linéaire quelle que soit l'instance. Il n'y a donc "plus" de pire des cas ou de meilleur des cas.

# CLASSES DE COMPLEXITÉ PROBABILISTES

Une classe de complexité regroupe les problèmes (langages) qui partagent une même propriété en lien avec le temps d'exécution ou l'espace mémoire requis pour les résoudre.

Nous parlerons ici de problèmes de décision mais en Théorie de la complexité, on préfère souvent parler de langage reconnaissable par un modèle de calcul (machines de Turing, machines RAM, automates cellulaires, machines quantiques etc.).

Lien entre les deux points de vue :

| Problème de décision  | Langage                                    |
|-----------------------|--------------------------------------------|
| Instances du problème | Langage $\mathcal{I} \subset \{0, 1\}^*$   |
| Instance positives    | Langage $\mathcal{I}^+ \subset \{0, 1\}^*$ |
| Résoudre le problème  | Reconnaître le langage $\mathcal{I}^+$     |

Reconnaître si  $w \in \{0, 1\}^*$  (ou  $\mathcal{I}$ ) est dans le langage  $\mathcal{I}^+$  revient à résoudre le problème de décision (beaucoup de détails sont omis dans cette description).

## DEFINITION

Un problème de décision est dit dans la classe  $RP$  (random polynomial-time) s'il existe un algorithme probabiliste  $\mathcal{A}$  tel que

$$x \text{ est une instance positive} \Rightarrow \Pr[\mathcal{A}(x) = 1] \geq \frac{1}{2}$$

$$x \text{ est une instance négative} \Rightarrow \Pr[\mathcal{A}(x) = 1] = 0$$

**Question 1.** Pour les problèmes dans  $RP$ , quelle réponse (0 ou 1) ne donne lieu à aucune erreur ? Justifiez.

**Question 2.** Donnez un exemple de problème vu en cours qui est dans  $RP$ .

## DEFINITION

Un problème de décision est dit dans la classe  $co - RP$  (complementary random polynomial-time) s'il existe un algorithme probabiliste  $\mathcal{A}$  tel que

$$x \text{ est une instance positive} \Rightarrow Pr[\mathcal{A}(x) = 1] = 1$$

$$x \text{ est une instance négative} \Rightarrow Pr[\mathcal{A}(x) = 0] \geq \frac{1}{2}$$

**Question 3.** Pour les problèmes dans  $co - RP$ , quelle réponse (0 ou 1) ne donne lieu à aucune erreur ? Justifiez.

**Question 4.** Donnez un exemple de problème vu en cours qui est dans  $co - RP$ .

## DEFINITION

Un problème de décision est dit dans la classe  $co - RP$  (complementary random polynomial-time) s'il existe un algorithme probabiliste  $\mathcal{A}$  tel que

$$x \text{ est une instance positive} \Rightarrow Pr[\mathcal{A}(x) = 1] = 1$$

$$x \text{ est une instance négative} \Rightarrow Pr[\mathcal{A}(x) = 0] \geq \frac{1}{2}$$

**Question 5.** Que pensez-vous de la valeur  $1/2$  dans les définitions précédentes ? Peut-elle être remplacée par une autre valeur ? Si oui, comment procéder ?

## DEFINITION

Pour un problème de décision  $\mathcal{D}$  et une instance  $x$  du problème, on note  $\chi_{\mathcal{D}}(x)$  la réponse (1=positive ou 0=négative) au problème. Un problème de décision  $\mathcal{D}$  est dit dans la classe  $ZPP$  (Zero error probability) s'il existe un algorithme probabiliste  $\mathcal{A}$ , de complexité (dans le pire des cas) polynomiale, retournant 0, 1 ou *FAIL* tel que

$$\forall x, \quad Pr[\mathcal{A}(x) = FAIL] \leq \frac{1}{2}$$
$$\forall x \quad Pr[\mathcal{A}(x) = \chi_{\mathcal{D}}(x) \text{ ou } \mathcal{A}(x) = FAIL] = 1$$

**Question 6.** Considérons un problème dans  $ZPP$  et un algorithme probabiliste  $\mathcal{A}$  pour le résoudre. Construire un algorithme  $\mathcal{B}$  qui n'échoue jamais et qui retourne toujours la bonne réponse.

**Question 7.** Soit  $p(n)$  la complexité de l'algorithme  $\mathcal{A}$  sur des instances de taille  $n$ . Donnez une borne pour la complexité moyenne de l'algorithme  $\mathcal{B}$ .

**Question 8.** A quelle famille d'algorithmes appartient l'algorithme  $\mathcal{B}$ ?

## DEFINITION

$ZPP$  = soit FAIL avec proba  $\leq 1/2$ , soit la bonne réponse

**Question 9.** Donnez un exemple de problème dans  $ZPP$  qui a été vu en cours.

**Question 10.** Montrez que  $ZPP \subseteq RP$ .

**Question 11.** Montrez que  $ZPP \subseteq co - RP$ .

**Question 12.** Montrez que  $ZPP = RP \cap co - RP$ .



# AUTRES ALGORITHMES PROBABILISTES

- Sélection de la médiane
- Problème des  $n$  reines
- Générateurs pseudo-aléatoires
- Paradoxe des anniversaires et fonctions de hachage
- protocoles cryptographiques probabilistes
- calcul d'enveloppes convexes
- optimisation stochastique (pour l'IA par exemple)
- Recuit simulé (chaînes de Markov)
- algorithmes génétiques
- ...